

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

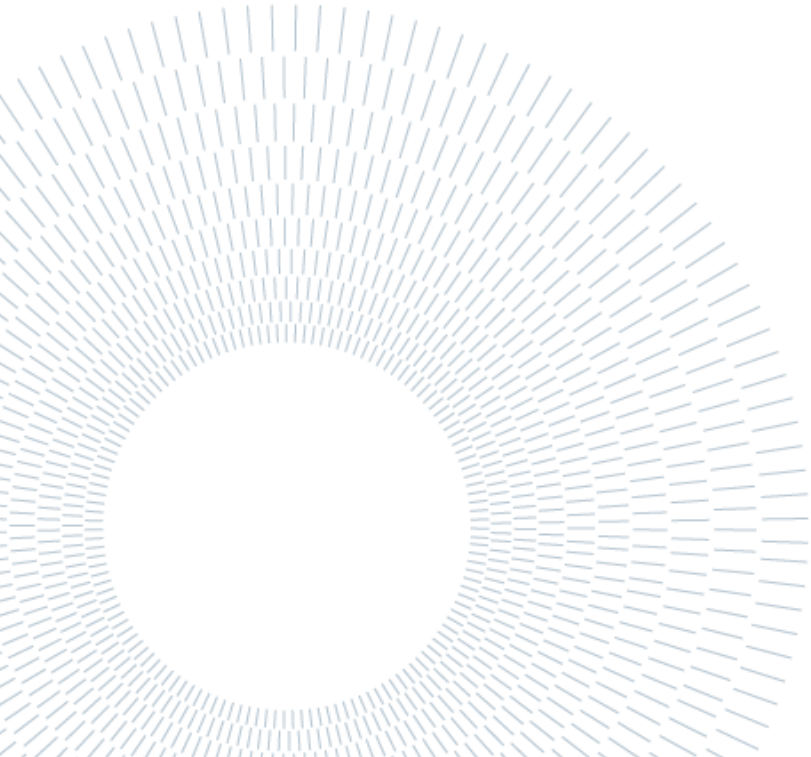
RASD

Requirements analysis and specification document

Best Bike Paths

Author: **Anna Cordioli – Tasnem El Koush**

Students: Anna Cordioli – 10891104
Tasnem El Koush - 10807328
Professor: Elisabetta Di Nitto
Academic Year: 2025-26



Contents

List of Figures v
List of Tables vi
1. Introduction 7
1.1 Purpose 7
1.1.1. Goals 7
1.2. Scope 9
1.2.1. World phenomena 10
1.2.2. Shared phenomena 10
1.3. Definitions, Acronyms, Abbreviations 14
1.4. Revision history 14
1.5. Reference Documents 14
1.6. Document Structure 14
2. Overall description 17
2.1. Product perspective 17
2.1.1. Scenarios 17
2.1.2. Class Diagram 21
2.2. Product functions 23
2.3. User characteristics 29
2.4. Assumption 29
3. Specific requirements 31
3.1. Product perspective 31
3.1.1. User Interfaces 31
3.1.2. Hardware Interfaces 31
3.1.3. Software Interfaces 31
3.1.4. Communication Interfaces 32
3.2. Functional Requirements 32
3.2.1. Requirements List 32
3.2.2. Use Case Diagrams 34
3.2.3. Use Cases 35
3.2.4. Goals-Requirements Mapping 49
3.3. Requirements 51
3.4. Design Constraints 51
3.4.1. Standards compliance 51
3.4.2. Hardware limitations 52

3.4.3. Any Other Constraint 52
3.5. Software System Attributes 52
3.5.1. Reliability 52
3.5.2. Availability 52
3.5.3. Security 52
3.5.4. Maintainability 53
3.5.5. Portability 53
4. Formal analysis using Alloy 54
4.1 Alloy model 54
4.2 Alloy result 64
5. Effort spent 66
6. References 66
6.1. References 66
6.2. Used Tools 66

List of Figures

Figure 1- Class diagram 21
Figure 2- State diagram for user registration and login 24
Figure 3 - State diagram for route searching 25
Figure 4 - State diagram for path visualization 25
Figure 5 - State diagram for start and stop recording 26
Figure 6 - State diagram for manual path insertion 26
Figure 7 - State diagram for automated detection 27
Figure 8 - State diagram for data review 27
Figure 9 - State diagram for visualization of My Trips 28
Figure 10 - State diagram for weather enrichment 28
Figure 11 - Unregistered User Abilities 34
Figure 12 - Registered User Abilities 34
Figure 13 - Sequence diagram for login 35
Figure 14 - Sequence diagram for registration 36
Figure 15 - Sequence diagram for trip recording 38
Figure 16 - Sequence diagram for search and visualize paths 39
Figure 17 - Sequence diagram for visualizing My Trips 41
Figure 18 - Sequence diagram for automated detection 43
Figure 19 - Sequence diagram for path insertion 45
Figure 20 - Sequence diagram for Review and Save/Publish Options 47
Figure 21 - Sequence diagram for weather enrichment 48

List of Tables

Table 1 - Goals description 9
Table 2 - World Phenomena description 10
Table 3 - Shared Phenomena description 13
Table 4 - Acronyms definition 14
Table 5 - Assumptions description 30
Table 6 - Requirements list 34
Table 7 – Goals-Requirements map 50

1. Introduction

Cycling has recently grown significantly, both as a means of daily transportation to support sustainable mobility and as a sport activity. However, cyclists often face difficulties in finding up-to-date and detailed information on route conditions or the availability of dedicated bike paths.

Furthermore, cyclists are interested in tracking their activities and comparing their performances based on distance, time or environmental conditions.

Best Bike Paths (BBP) is intended to address these needs. The application will provide a centralized system for recording cycling activities, collecting and sharing information on bike paths, reporting problems, and visualize optimal routes. Through active user interaction, BBP aims to build a information network on the quality and safety of cycling routes.

1.1 Purpose

This document is the Requirement Analysis and Specification Document (RASD) that serves as a comprehensive reference for the development team involved in implementing the system.

This document provides a clear specification of the platform's functionalities and constraints for all stakeholders involved in the project, including:

- The biker's association initiating the project
- The development team responsible for system implementation
- Active cyclists who record their trips, insert path information, and actively contribute to the platform.
- Non-registered and registered users (general public) who access the platform to consult the available bike paths information.

1.1.1. Goals

The main goals of the BBP system are summarized in the following table, organized by user perspective:

Goal	Description
From any user perspective (browsing and searching paths)	
G01	Allow U to register to the platform
G02	Allow U to search for bike paths by specifying an origin and a destination
G03	Allow U to view search results ranked by score when multiple paths exist
G04	Allow U to visualize bike path information published by others
G05	Allow U to visualize bike paths on a map between two specified points
From registered user perspective (account management)	
G06	Allow RU to login and logout
G07	Allow RU to insert and modify personal information
From registered user perspective (device permission and settings)	
G08	Allow RU to grant GPS access to the system (required for data collection in case of trip recording and automated detection)
G09	Allow RU to grant optional access to accelerometer and gyroscope sensors for obstacle detection
G10	Allow RU to enable/disable automated data collection mode necessary to detect cycling activity
From registered user perspective (manual trip recording)	
G11	Allow RU to start/stop recording their bike trips
G12	Allow RU to review the data gathered of the trip recorded and correct them if needed before confirming them
G13	Allow RU to save manually recorded trip data as a personal trip
G14	Allow RU to publish manually recorded trip data as a bike path
G15	Allow RU to discard manually recorded trip data
From registered user perspective (personal trip management)	
G16	Allow RU to access their trip history
G17	Allow RU to view statistics about each recorded trip (distance, average speed, duration)

G18	Allow RU to view meteorological information associated with their trips when available
G19	Allow RU to publish an existing trip as a bike path
From registered user perspective (manual path information insertion)	
G20	Allow RU to manually insert bike path information by specifying street names and their status (optimal, medium, sufficient, requires maintenance), and other information such as distance and duration
G21	Allow RU to manually indicate the presence of obstacles during the submission of the path information
G22	Allow RU to save manually inserted path information as a personal trip
G23	Allow RU to publish manually inserted path information as a bike path
G24	Allow RU to discard manually inserted path information
From registered user perspective (automated path information collection)	
G25	Allow RU to review the data gathered (origin, destination, obstacles, street names, path status) automatically and modify them if needed before confirming them
G26	Allow RU to make automatically collected path information publishable
G27	Allow RU to save the data gathered automatically as a personal trip
G28	Allow RU to discard the data gathered automatically

Table 1 - Goals description

1.2. Scope

Best Bike Paths (BBP) system is a platform that aims to improve cycling experience by providing detailed information about bike paths and supporting users in planning, tracking and evaluating their rides. The system enables registered users to record their trips with detailed statistics such as distance, average speed, and duration. When available, the platform enriches trip data with meteorological information from external services, providing environmental context to record activities.

Registered users can contribute to the platform in three different ways:

- *manual insertion*: RU specify street names, path status (optimal, medium, sufficient, requires maintenance) and report the obstacles on the path;

- *manual recording*: RU manually start and stop the trip recording after enabling the device's GPS. The system tracks the route and identifies potential obstacles through sensor data analysis if enabled. RU can review, confirm, or correct those collected data;
- *automated mode*: the system automatically collects data from device sensors (GPS, accelerometer, and gyroscope) while they bike. In automated mode, the system detects the user's movement, tracks the route via GPS, and identifies potential obstacles through sensor data analysis if enabled. RU can review, confirm, or correct automatically collected data.

After collecting or inserting path information through any of these three methods, users can always save the data as a personal trip, publish it as a bike path to share with the community, do both simultaneously, or discard it entirely.

Any user, registered or not, can search for bike paths between specified origin and destination points. The system returns available paths ranked by score based on their status and effectiveness in connecting the two locations. Users can then choose to visualize these paths on a map.

By facilitating the sharing of paths within the cycling community, BBP aims to provide useful information that helps cyclists make decisions about their routes.

1.2.1. World phenomena

WP ID	Description
WP01	Weather conditions exist in the environment (temperature, wind, precipitation)
WP02	Users decide to go biking
WP03	Physical bike path exists with their characteristics (obstacles, surface conditions)
WP04	Mobile devices have GPS, accelerometer, and gyroscope sensors
WP05	Users have the knowledge to evaluate bike path conditions
WP06	Users cycle with their device

Table 2 - World Phenomena description

1.2.2. Shared phenomena

SP ID	Description	Controller	Observer
User registration			
SP01	U registers to the system	U	BBP

User authentication			
SP02	RU logs into the platform	RU	BBP
SP03	RU logs out from the platform	RU	BBP
Account management (RU)			
SP04	RU modifies personal information	RU	BBP
Path search and Visualization (U)			
SP05	U specifies origin and destination for bike path search	U	BBP
SP06	BBP displays a list of bike paths ranked by score	BBP	U
SP07	BBP displays the score for each path to the user	BBP	U
SP08	U requests to visualize bike paths on a map	U	BBP
SP09	BBP displays multiple paths on the map	BBP	U
SP10	U browses published path information	U	BBP
Device permissions and settings (RU)			
SP11	RU grants GPS access to the system	RU	BBP
SP12	RU grants access to accelerometer and gyroscope sensors	RU	BBP
SP13	RU enables/disables automated data collection mode	RU	BBP
Manual trip recording (RU)			
SP14	RU starts/stops manual trip recording	RU	BBP
SP15	BBP assigns suggested path status based on detected obstacles and route characteristics	BBP	BBP
SP16	BBP displays collected trip data for review with suggestions (origin, destination, street names, obstacles, suggested path status) for review	BBP	RU
SP17	RU reviews and corrects manually recorded trip data	RU	BBP

SP18	RU saves manually recorded trip data as a personal trip	RU	BBP
SP19	RU publishes manually recorded trip data as a bike path	RU	BBP
SP20	RU discards manually recorded trip data	RU	BBP
Personal trip management (RU)			
SP21	RU requests to view trip history	RU	BBP
SP22	BBP displays list of past recorded trips	BBP	RU
SP23	RU selects a trip to view details	RU	BBP
SP24	BBP displays detailed trip information (statistics, route, meteorological data if available)	BBP	RU
SP25	RU publishes an existing trip as a bike path	RU	BBP
Manual path information insertion (RU)			
SP26	RU manually inserts bike path information specifying street names, path status	RU	BBP
SP27	RU manually indicates obstacles during path insertion	RU	BBP
SP28	RU saves manually inserted path information as a personal trip	RU	BBP
SP29	RU publishes inserted path information as a bike path	RU	BBP
SP30	RU discards manually inserted path information	RU	BBP
Automated path detection (RU)			
SP30	User's device sensors send GPS coordinates to BBP	Device sensors	BBP
SP31	User's device sensors send accelerometer and gyroscope data	Device sensors	BBP
SP32	BBP detects that the user is biking and starts data collection	BBP	RU
SP33	BBP detects that the user stopped biking and ends data collection	BBP	RU

SP34	BBP notifies RU that data collection is complete	BBP	RU
SP35	BBP assigns suggested origin, destination, and street names of the route based on GPS tracking	BBP	BBP
SP36	BBP assigns a path status based on detected obstacles and route characteristics from native mapping service	BBP	BBP
SP37	BBP displays automatically collected path data for review with suggestion for review	BBP	RU
SP38	RU reviews and corrects automatically collected path information	RU	BBP
SP39	RU can makes confirmed collected path information publishable as a bike path	RU	BBP
SP40	RU discards automatically collected path	RU	BBP
SP41	RU saves automatically collected path as a trip	RU	BBP
Meteorological data integration			
SP42	BBP requests meteorological data from external service	BBP	External service
SP43	External weather service provides data to BBP	External service	BBP
SP44	BBP stores trip data enriched with meteorological information	BBP	BBP
SP45	BBP displays meteorological information associated with recorded trip to RU	BBP	RU
Bike path scoring			
SP46	BBP assign a path score based on path status and effectiveness	BBP	BBP
SP47	BBP publishes bike path with calculated score	BBP	Community

Table 3 - Shared Phenomena description

1.3. Definitions, Acronyms, Abbreviations

Acronym	Definition
BBP	Best Bike Path
RASD	Requirements Analysis Specification Document
RU	Registered User
SMTP	Simple Mail Transfer Protocol
SP	Shared Phenomena
U	Any user
WP	World Phenomena

Table 4 - Acronyms definition

1.4. Revision history

- Version 1.0: Initial Release →23/12/2025

1.5. Reference Documents

- Slide and lesson notes
- Alloy documentation
- Assignment RDD AY 2025-2026
- Peer review feedback

1.6. Document Structure

1. **Introduction**
   This section presents the main objectives of the project and clarifies its overall purpose. It defines the project's scope through an analysis of the relevant context and shared phenomena. Key terminology, acronyms, and abbreviations are listed to ensure clarity.
2. **Overall description**
   This part outlines the general view of the system and its environment. It illustrates how the product fits within its broader context by describing representative scenarios and providing domain models, such as class and state diagrams. The main system functionalities are summarized, together with information about the target users, their needs, and expectations. It also identifies assumptions, dependencies, and constraints that influence the system's design and behaviour.
3. **Specific Requirements**

This section details all requirements necessary for development. It includes descriptions of external interfaces—covering user, hardware, software, and communication aspects—and defines the main functional requirements through use cases and related diagrams (use case, sequence, or activity). Performance goals, design limitations, and software quality attributes such as reliability, security, maintainability, and portability are also specified.

4. **Formal analysis using Alloy**
   This section introduces the goals behind the formal modelling activity and explains the structure of the Alloy model. It discusses what the model verifies, which properties are checked, and why these results are relevant to the system's correctness and consistency.
5. **Effort spent**
   This part summarizes the distribution of workload within the team, specifying the number of hours contributed by each member in preparing this document and performing related activities.
6. **Reference**
   This final section lists all external materials, technical papers, standards, and online resources referenced throughout the document.

2. Overall description

2.1. Product perspective

2.1.1. Scenarios

1. **Registration:** Laura is a 28-year-old working professional who has recently moved to Milan and wants to start going to work by bicycle. She heard about BBP from a colleague and decides to explore the platform. She notices that she can continue as a guest user and search for bike paths without creating an account, but she prefers to access the advanced features reserved for registered users. Laura selects the "Register" button and fills out the form of registration with her full name, email address, and a password. After submitting the information, the system checks that the email is not already in the database and that the password meets the minimum-security requirements. All checks pass successfully, and, within moments, the system displays the message: "Welcome to Best Bike Paths!". Laura is automatically authenticated and can now access the features reserved for registered users.
2. **Login:** A few months after registering, Laura notices that she has been automatically logged out of the application for security reasons after a period of inactivity. She selects the "Login" button and enters her email and password. The system checks the credentials in the database and, finding them correct, authenticates Laura by redirecting her to the main screen where she can once again access all the features.
3. **Manual Recording:** Edward, a 35-year-old cycling enthusiast, wants to monitor his performance during training sessions. One morning, he opens the BBP app and selects **"**Start Trip Recording". Since he has already granted GPS access and sensors permissions from his profile (accelerometer and gyroscope for obstacle detection), the recording starts immediately in background. During the ride, the system records GPS data and through sensor analysis, detects potential obstacles on the road. At the end of the route, Edward selects "Stop Trip Recording," and the system displays a trip summary with all the data collected: duration, distance, average speed, elevation gain, obstacles detected, suggested origin, suggested destination and a suggested path status. Edward reviews suggested data, and since he only wants to track his personal performance without sharing with the community, he selects "Save as Trip" without publishing. The system creates the Trip object and saves everything to his personal history. Edward can now view his trip in the "My

Trips" section and, if he changes his mind later, he can publish it as a bike path to help other cyclists.

4. **Automatic Detection and Publication:** Jennifer is a freelance photographer who cycles regularly but barely follows the same route. She travels to various locations across the city, so she realized that manually starting and stopping trip recording of each path would be tedious given how frequently she moves around the city. For this reason, when she installed BBP, she enabled the "Automatic Activity Detection" in the app settings and granted access to GPS and sensors, allowing the app to monitor her movement in the background, and automatically detect whenever she bikes.
   One morning, Jennifer cycles to a client in Porta Romana. The system detects her activity automatically and starts data collection: GPS coordinates are tracked continuously while accelerometer and gyroscope data monitor for potential obstacles. During the 20-minute ride, two potential obstacles are detected. Upon arrival, when her speed drops to zero for some time, the system stops recording and automatically ends the data collection session. Jennifer receives a notification summarizing the trip with suggested data assigned by BBP. During her lunch break, she opens the notification, and she sees the details of her activity, showing:
   - Suggested origin: Viale Zara 58
   - Suggested Destination: Porta Romana 6,
   - Distance: 5.1 km
   - Duration: 20 min
   - Detected obstacles: 2
     - Pothole (km 2.1, V.le Gadio 8)
     - Debris (km 5.0, Via Giulianova 1)
   - Suggested path status: "Medium"

   Jennifer trusts the accuracy of her device's sensors and does not feel the need to correct the suggested data. She confirms the collected information and decides to save this activity as a personal trip and publish it as a bike path to contribute to the community.
5. **Manual Path Insertion and Publication:** Giovanni is an experienced cyclist who knows a beautiful bike route between two points. He decides to share this knowledge with the community. He opens the BBP app and selects "Insert a Route", and he inserts the following information:
   - Origin and destination points
   - Street names included in the route
   - Path status ("Optimal")
   - Obstacles encountered (1 pothole on Via Alemagna)
   - Duration
   - Distance

   After submitting the form, he decides only to publish without saving it as a personal trip and the systems publishes the path's information and assigns score

of 8,5 out of 10 based on the path status and obstacles encountered and route characteristics from native mapping service.

6. **Reviewing and publishing automatically collected data:** Roberto is an experienced cyclist who uses BBP with automatic detection enabled. Last Saturday, he cycled through a new park route, and the system automatically collected data, detecting 3 potential obstacles. He received the notification and immediately opens it to review the collected data.
   He corrects the suggested destination address from " Parco Sempione" to "Ingresso Nord Parco Sempione", then he carefully reviews each obstacle individually:
   - Pothole (km 2.1, V.le Gadio 8): "Yes, pothole" → Confirms
   - Debris (km 5.0, Via Giulianova 1): "No, just a speed bump" → Rejects

   At the end he modifies the suggested path status from "Medium" to "Optimal" and decides to save it as a trip. At the end of day, he decided to help the community and publishes the saved trip, so the system calculates a quality score for this route and makes it visible to all BBP users.
7. **Browsing path (with and without registration):** Alice, a potential new user who hasn't registered yet, is at home and wants to visit her boyfriend. However, all public transports are unavailable due to a strike, so she decides to look for a cycling alternative since her brother owns a bike. She opens Best Bike Paths (BBP) and notices that, even without an account, the interface allows her to search for routes. She quickly enters "Piazza Leonardo da Vinci, 1" as her origin and "Piazza Duomo, 1" as her destination. Within seconds, BBP processes her request and displays several possible paths, ordered by their quality score. Alice notices that Path A has the highest score and selects it to view the details. She sees that the path is rated as "Optimal" and reports no significant obstacles, so she decides to take her brother's bike and head to her boyfriend's house.
8. **Visualizing and comparing different paths:** Carol is an experienced cyclist who uses BBP to plan her daily commute. After logging in, she enters her home address as the origin and her workplace as the destination. The BBP system identifies three feasible routes between the two points. To better compare the routes visually, Carol taps "View on Map". BBP open interactive map showing all three bike paths overlaid on the geographical map. He can also tap on each path to view the detailed information. Path X is the shortest at only 8 km, but it has a score of 6.5 due to its "Medium" status and the presence of three potholes. Path Y is longer (10 km) but achieves a score of 9.2, thanks to its "Optimal" status and the absence of obstacles. Finally, Path Z is the longest (12 km) with a score of 8.0 and a "Sufficient" status. After reviewing the comparison, Carol decides that the extra 2 km of distance (Path Y) is a small price to pay for maximum safety and the best maintenance conditions.
9. **Trip statistics and integration with meteorological data:** David recorded a trip last week using BBP's automated mode on his mobile device. He now wants to review his performance. After logging into the BBP system, he navigates to the "My Trips"

section and selects the trip from last Tuesday at 3:00 PM. The system displays the basic statistics, including a total distance of 25 km and an average speed of 20 km/h, and duration of 75 minutes - slower than his usual pace. Puzzled, David notices that a “View Meteorological Data” option is available. This feature appears only when BBP has successfully retrieved weather data from its integration with an external meteorological service. Curious, he clicks on it, and the system displays environmental conditions during his ride:

- Weather Condition: Light Rain
- Temperature: 12 °C
- Wind Speed: 15 km/h.

David immediately understands that the wind and rain had directly affected his average speed, providing valuable context for interpreting his performance data.

2.1.2. Class Diagram

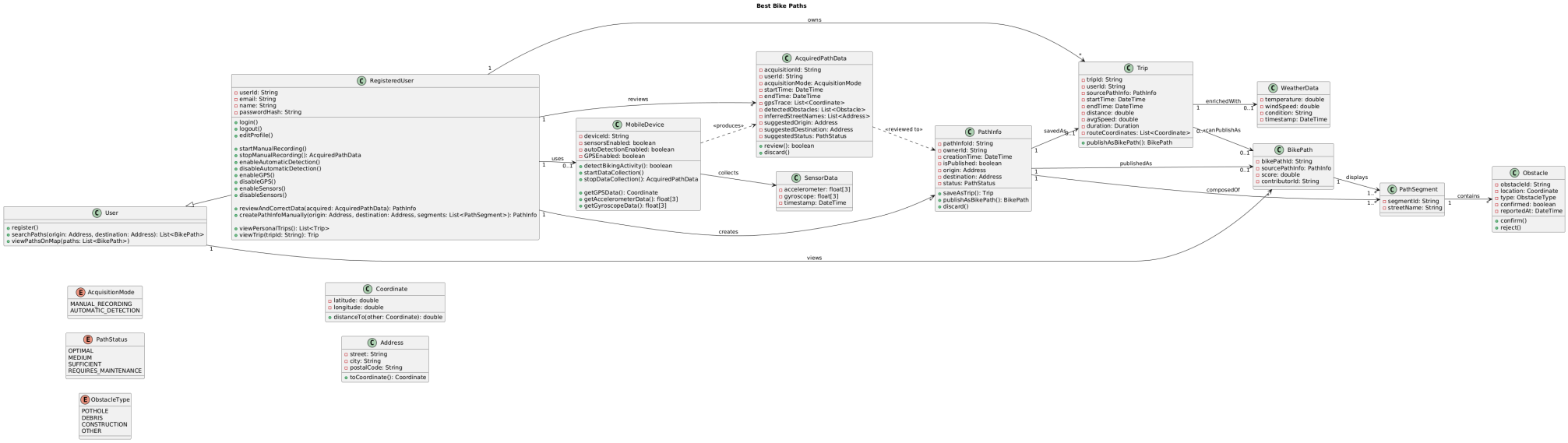


Figure 1- Class diagram

2.1.2.1 Main classes and diagram description

The diagram is centred around the **User** class, which is specialized through inheritance into **Registered User**. This distinction is fundamental to the system's access control and functionality. The application supports two types of users: guest users (represented by the base User class) can only search and view published bike paths, while registered users have access to the complete functionality including recording trips, managing their history, and publishing routes to the community.

Path-Related Classes Hierarchy

The path-related classes follow a clear workflow with distinct states and purposes:

- **AcquiredPathData**

  Represents **raw data collected during recording**, whether through:

  - **Manual mode**: User explicitly starts and stops trip recording.
  - **Automatic Detection Mode**: Works in the background once enabled by the user. The system monitors sensors to recognize when the user is cycling and automatically start recording and collecting data without any user intervention. When the system detects that cycling has stopped, it automatically ends the recording session.

  Sensors access and permission: Regardless of the acquisition mode, registered users must grant the application access to the device's location services to collect GPS coordinates and infer street name along the route. Access to motion sensors (accelerometer and gyroscope) is optional but recommended. These sensors, represented as three dimensional arrays (float[3] containing x, y, z coordinates) are used to keep track of any significant movement of the device itself that suggests the presence of obstacles.

  The result in any case is an **AcquiredPathData** object containing GPS traces, detected obstacles (if sensors are enabled), inferred street names, and system-generated suggestions for the route's origin, destination, street names, obstacles detected and path status. The path status is assigned based on detected and the route and route characteristics. All this information is considered preliminary and requires user verification before it can be used.

- **PathInfo**

  Represents verified data created through three methods:

  - By reviewing and correcting AcquiredPathData from manual recording
  - By reviewing and correcting AcquiredPathData from automatic detection
  - Through direct manual input of a known route without recording it first.

In all cases, the information are human verified: obstacles are confirmed, street names validated, and route quality established.

PathInfo acts as intermediate processing stage before the user decides the final destination of their data. They can save it as a personal Trip for performance tracking, publish it as a BikePath to share with the community, or do it both simultaneously, or discard all the information.

- **Trip**

  Created when PathInfo is saved privately to "My Trips". Trips focus on performance metrics: distance, average speed, duration. The trip can be enriched with weather data when available. These records accessible only to the user who created them and serves as a personal activity history.

  A key feature is that Trips maintain a reference to their source PathInfo, which contains all the verified route details including path segments and confirmed obstacles. This design enables delayed publication: users can publish any Trip as a BikePath at any time even long after they were initially saved. This is why the publish() method appears in the Trip class, allowing users to share their personal recordings with the community whenever they choose.

- **BikePath**

  Represents a **complete and published route** available on the platform. This is the only type of path visible to unregistered users.

  They can be created either directly from PathInfo after reviewing collected or manually entered data, or by publishing an existing Trip. Once published anyone can search for bike paths between any two points in the city. Every BikePath is assigned a quality score, calculated by the system based on path status and obstacle presence and route characteristics, helping users choose the most suitable route for their needs.

2.2. Product functions

U functionalities:

- **User access management**: Users once they installed the application, they have the option to access BBP without creating an account. If they prefer to take a quick look at the platform, they can continue as a guest. However, guest users will have access to limited features. For potential users wishing to access full functionalities, BBP implements a standard mechanism to allow cyclists to create personal accounts. During registration users, must provide a valid email address and a password. The system validates the email format, checks for email uniqueness in the database, and securely stores password using hashing. Upon successful registration, users gain access to personalized features. The login process

authenticates users through their username and password, granting them access to the platform.

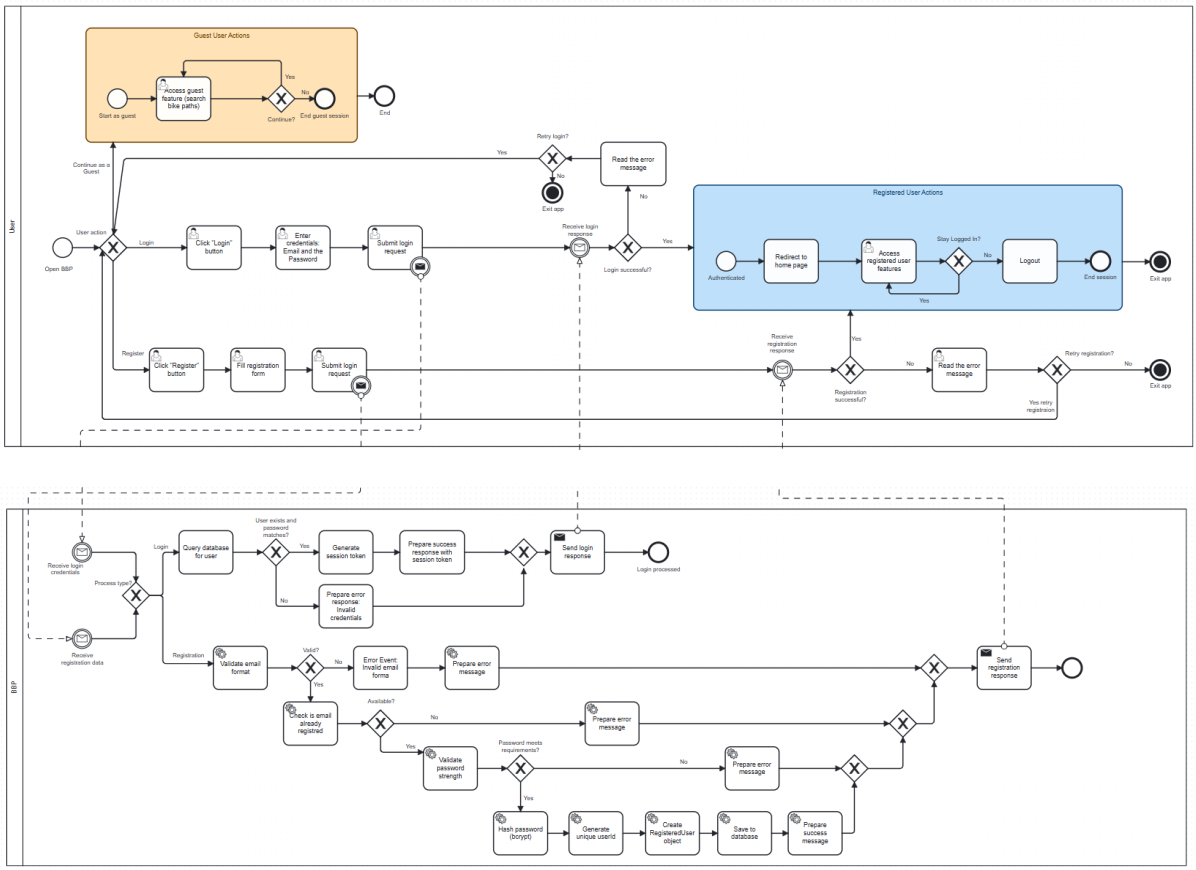


Figure 2- State diagram for user registration and login

- **Search a route between two points:** Any user can search for bike paths by specifying the address of the origin and destination points. The system validates the addresses using geocoding services to convert them into geographic coordinates. Once validated, BBP queries the database for available published path information connecting the two locations. If matching paths are found, the system sorts the results by score in descending order. If no paths are found, the system informs the user.

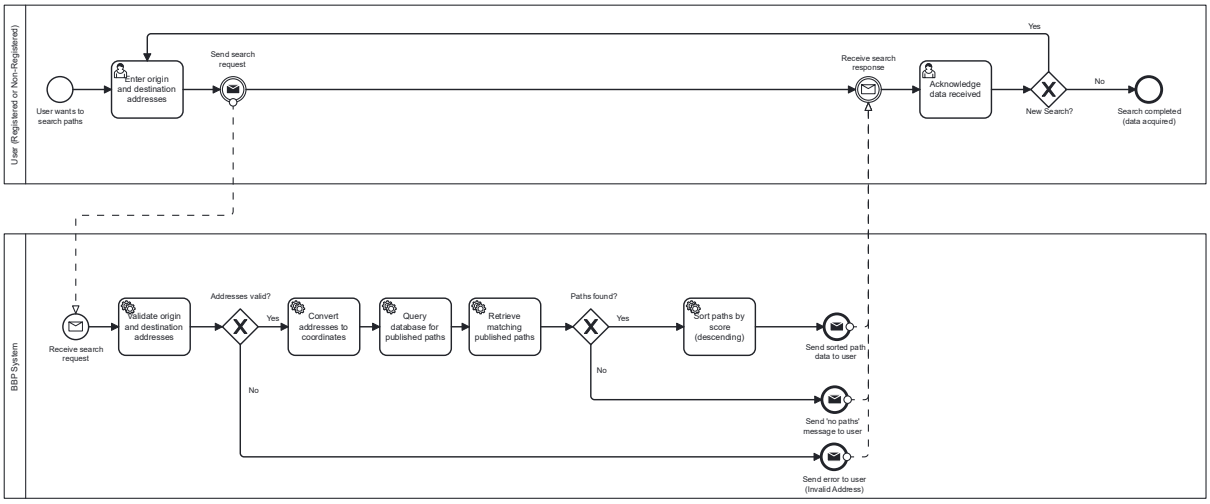


Figure 3 - State diagram for route searching

- **Path Visualization:** After receiving the ranked search results, users can request to visualize the paths on a map. The system integrates with the device's native mapping framework to render bike paths overlaid on a geographical map. This provides users an immediate visual overview of the available paths to facilitate quick comparison. From the map visualization, they can select any path to view detailed information about it, including its status, distance, and obstacles with their location.

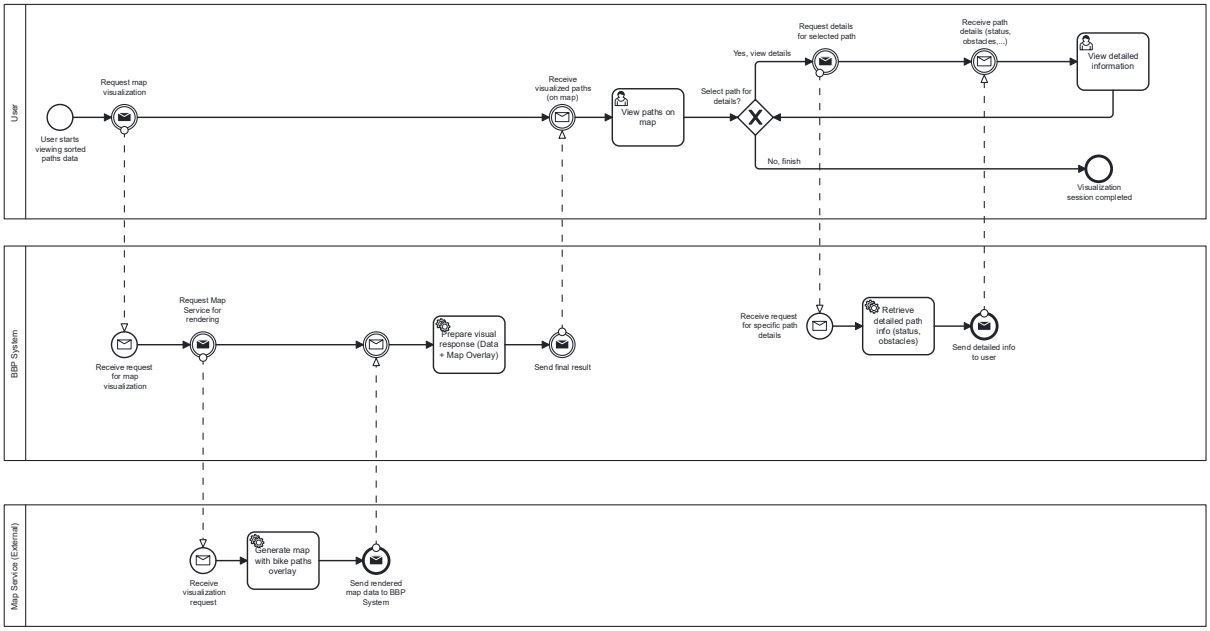


Figure 4 - State diagram for path visualization

RU functionalities:

- **Manual Trip Recording:** RU can manually start/stop trip recording. GPS and optional sensor data are collected. After stopping, BBP process the data, calculates

statistics (distance, average speed, duration) and assigning a suggested path status based on obstacles and route characteristics. The system then displays all collected data with suggestions for the user to review and correct. After review, users have can then:

1. Save as personal Trip (system enriches with weather data if available),
2. Publish as BikePath (system calculates quality score),
3. Save & publish simultaneously (with weather enrichment and score calculation),
4. Discard the data.

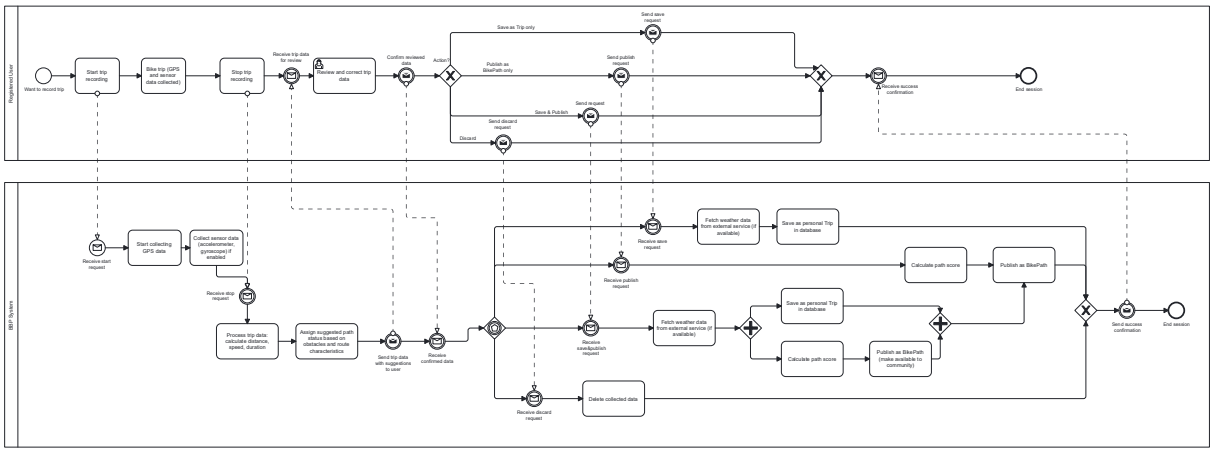


Figure 5 - State diagram for start and stop recording

- **Manual path insertion:** RUs have the possibility to manually insert a path they have travelled and provide all relevant details to help other cyclists improve their route search experience. Furthermore, they can report obstacles encountered on the path.

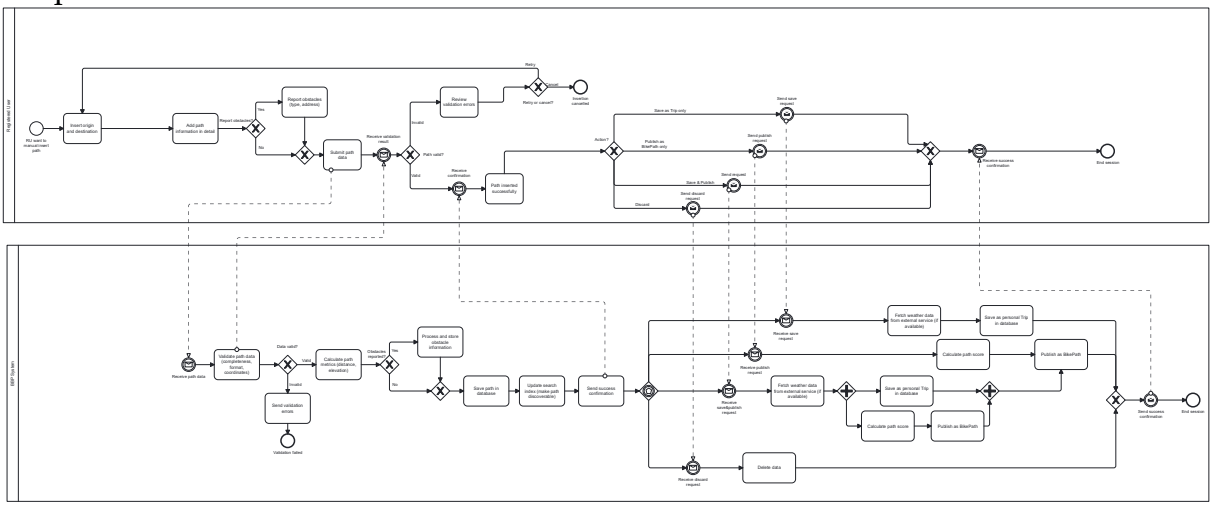


Figure 6 - State diagram for manual path insertion

- **Automatic Cycling detection:** If enabled by the RU, the system automatically detects when the user is cycling (via GPS speed monitoring) and collects GPS and

sensor data (accelerometer and gyroscope if enabled) in background. After trip ends user can decide to discard immediately if not interested or proceed to review and correct the data. After review, user can save as Trip, publish as BikePath, or save and publish simultaneously.

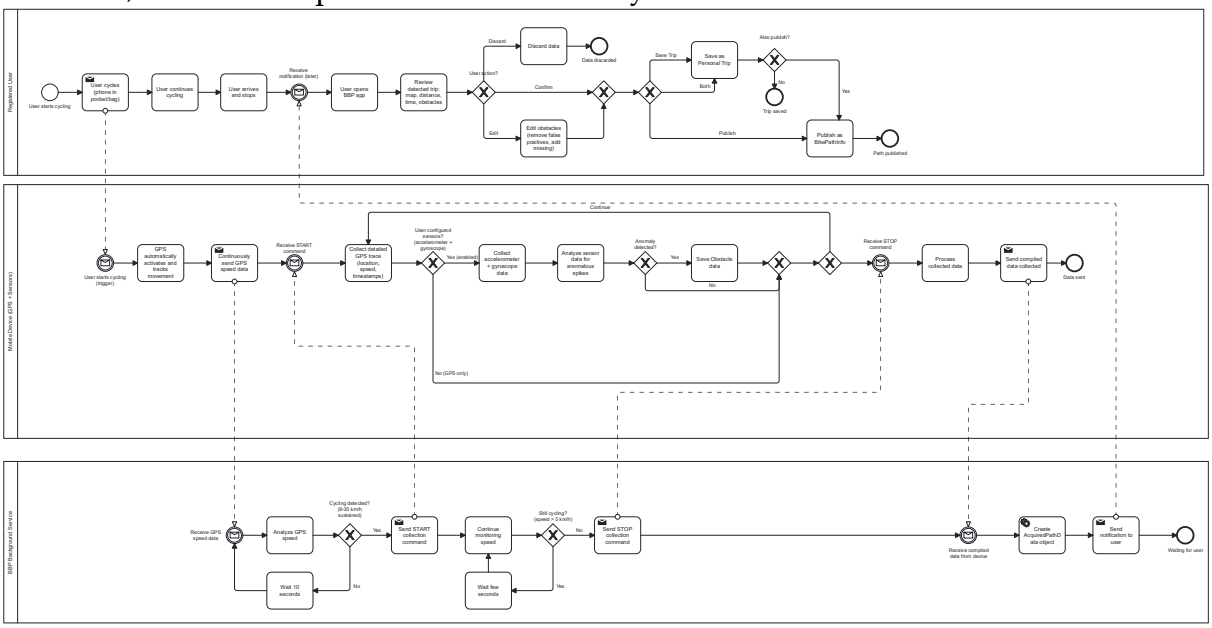


Figure 7 - State diagram for automated detection

- **Automated data review and confirmation**: After a trip has been automatically recorded or detected, the RU can review the details: suggested origin and destination, inferred street names, assigned path status, and a list of detected obstacles with their locations. The RU can review each obstacle individually and confirm or reject it. The RU can modify any detail including origin, destination, street names, path status, and obstacle positions. Afterwards, the RU can choose to publish the path, allowing everyone to benefit from their reports, or keep it private (not in this diagram).

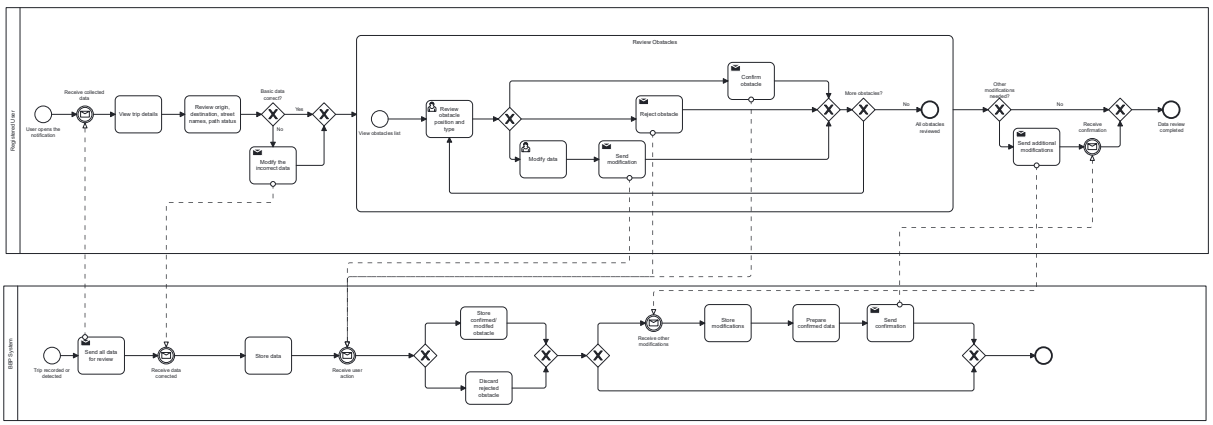


Figure 8 - State diagram for data review

- **Visualization of my trips:** The RU can access a tab called 'My Trips,' where they can view all recorded trips—both automatic and manual—along with detailed information for each trip, such as date, speed, distance covered, duration, and any obstacles encountered.

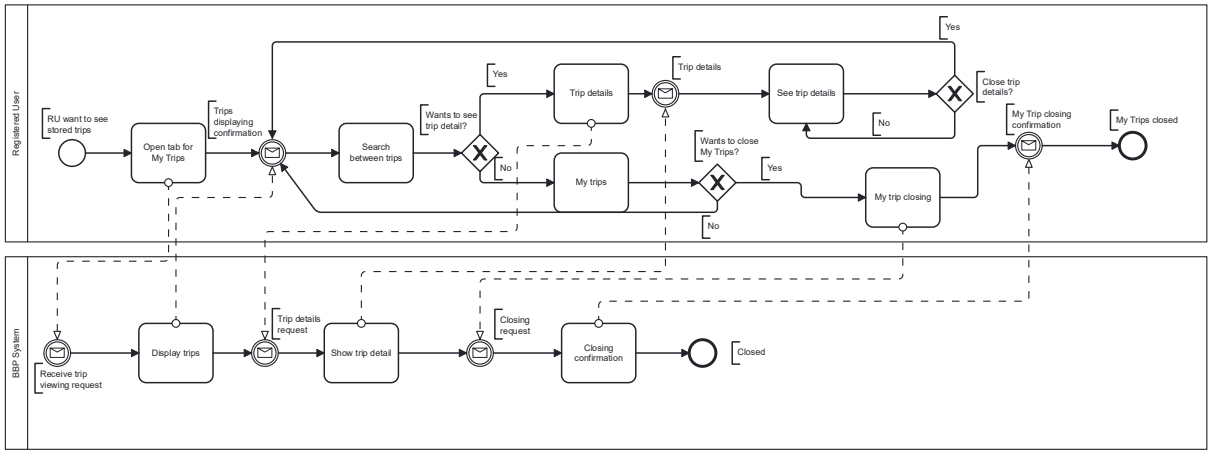


Figure 9 - State diagram for visualization of My Trips

- **Weather information enrichment**: When a trip is decided to save as a trip, the trip can be enriched with weather information, such as temperature, precipitations, and wind speed at the date and time of the trip. Availability of this data depends on the external service providing it.

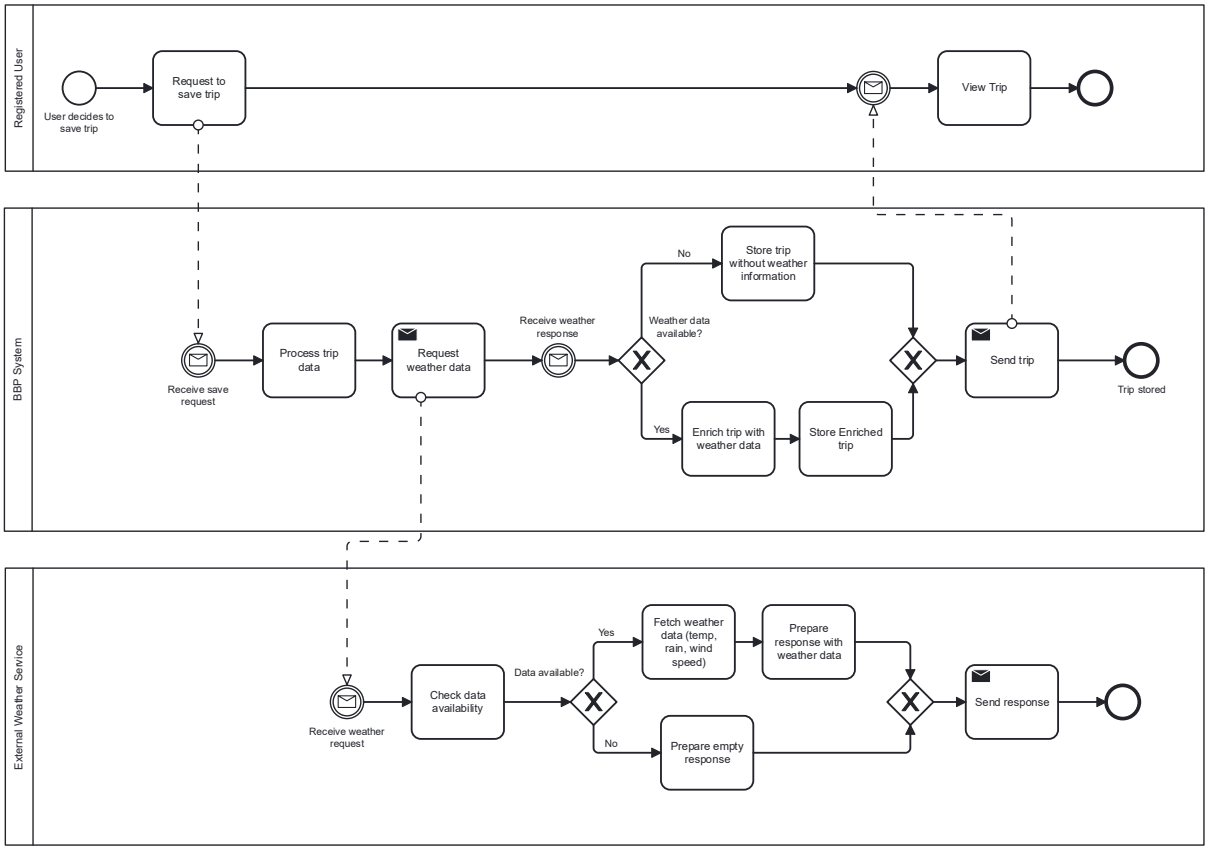


Figure 10 - State diagram for weather enrichment

2.3. User characteristics

The system is designed to be used by two main actors: users and registered users.

Users: are all individuals who access the system without registration:

- They can search for the best bike paths between two points.
- They can view and explore the suggested paths directly on a map.

Registered users: are users who register in the system to record, analyse, and share their bike trips:

- They are the core contributors to the BBP system, as their recorded trips continuously enrich the database.
- They can record their rides manually or automatically through mobile devices.
- They can store their trips to keep track of their activities.
- They can review, confirm, or correct automatically detected issues (e.g., potholes, maintenance needs).
- They can publish their validated data, making it available to the entire BBP community.
- They can access detailed trip statistics, including total distance, average speed, and performance metrics.
- They can benefit from enriched trip information, such as weather conditions and wind speed, provided by external services.

2.4. Assumption

Assumption	Description
A01	RU must have a mobile device
A02	GPS is available and provides accurate location data for path tracking
A03	Devices are equipped with an accelerometer and gyroscope sensors, which can provide data for obstacle detection when user permissions are granted.
A04	An external service is available to retrieve weather data based on time and GPS coordinates of a trip.
A05	When entering data manually about obstacles, path status, the system trusts user-provided information.
A06	BBP integrates with native mobile map service for route visualization.

A07	User's mobile device has a stable internet connection for Searching and Publishing bike paths

Table 5 - Assumptions description

3. Specific requirements

This chapter provides detailed specifications for the requirements previously mentioned that may require additional clarification for the development team's implementation.

3.1. Product perspective

3.1.1. User Interfaces

BBP is implemented as a mobile-based application for mobile devices. The UI follows modern mobile and user-friendly design principles with responsive layouts that adapt to different screen sizes and respects platform-specific guidelines for both iOS and Android.

3.1.2. Hardware Interfaces

BBP will be accessible from all mobile devices equipped with GPS and capable of capturing data from embedded sensors. The mobile devices must provide access to GPS, accelerometer, and gyroscope data to support automatic trip recording and obstacle detection.

3.1.3. Software Interfaces

BBP functions as a standalone mobile application with the following external dependencies:

- **External Weather Service**: BBP retrieves meteorological data (temperature, wind speed, weather conditions) through the device's native weather APIs based on GPS coordinates and timestamps to enrich a recorded trip information when available.
- **Mobile Sensor APIs**: The application interfaces sensor frameworks to access GPS, accelerometer, and gyroscope data for route tracking and automated obstacle detection.
- **Map Service**: BBP integrates with map service through their native mapping frameworks for geocoding, address resolution, and path visualization.

3.1.4. Communication Interfaces

BBP will communicate with users through mobile push notifications delivered directly to the mobile device in use. In addition, the system will use the SMTP protocol to send email notifications when required, such as for registration confirmation and password recovery.

3.2. Functional Requirements

3.2.1. Requirements List

ID	Requirement
All users – Browsing and searching	
R01	BBP enables users to register an account.
R02	BBP enables users to search a bike path between an origin and a destination.
R03	BBP displays bike paths on a map.
R04	BBP displays a list of paths ranked by their score when multiple paths are available.
R05	BBP displays the score for each bike path.
Registered users – Account management	
R06	BBP allows RU to log in.
R07	BBP allows RU to log out.
R08	BBP enables RU to view and edit their profile information.
Registered users – Device permissions and settings	
R09	BBP enables RU to grant GPS access to the system.
R10	BBP enables RU to grant access to accelerometer and gyroscope sensors.
R11	BBP enables RU to enable/disable automated data collection mode.
Registered users – Manual trip recording	
R12	BBP allows RU to start/stop recording of their cycling trips for personal activity tracking.
R13	BBP collects GPS data during manual trip recording.

R14	BBP collects accelerometer and gyroscope data during manual trip recording when sensors are enabled.
Registered users – Personal trip management	
R15	BBP provides statistics for each recorded trip including total distance, average speed, and performance metrics.
R16	BBP maintains a historical record of all trips recorded.
R17	BBP enables RU to publish an existing trip as a bike path.
Registered users – Manual path information insertion	
R18	BBP allows RU to manually insert bike path information by specifying street addresses.
R19	BBP enables RU to assign a path status using predefined categories (optimal, medium, sufficient, requires maintenance).
R20	BBP allows RU to report the presence of relevant obstacles (e.g., potholes) along paths.
Registered users – Automated path information collection	
R21	BBP automatically detects when the user is cycling based on movement speed.
R22	BBP collects GPS data from the mobile device to reconstruct the followed path.
R23	BBP acquires data from the mobile device's accelerometer and gyroscope when enabled to detect obstacles.
R24	BBP automatically stops data collection when cycling activity is no longer detected.
Registered users – Data processing and review (from automated detection or manual recording)	
R25	BBP processes collected GPS and sensor data to calculate trip statistics
R26	BBP suggests path information (origin, destination, street names, status) based on collected data
R27	BBP displays collected data with all the suggestions assigned to RU for review.
R28	BBP allows RU to review and correct all the suggested data gathered.
R29	BBP allows RU to reject or correct each obstacles detected.
R30	BBP allows RU to confirm collected data information.

Registered users – Save, publish, and discard actions	
R31	BBP enables RU to save path information as a personal trip.
R32	BBP enables RU to publish path information as a bike path to share with the community.
R33	BBP enables RU to discard path information without saving or publishing.
System- Meteorological data	
R34	BBP retrieves meteorological data from external service when saving a trip, using timestamp and GPS data.
R35	BBP enriches trip with weather information (conditions, temperature, wind speed) if available.
System – Path scoring and publication	
R36	BBP computes a path score based on path status and obstacles.
R37	BBP makes published bike paths searchable and visible to all users.

Table 6 - Requirements list

3.2.2. Use Case Diagrams

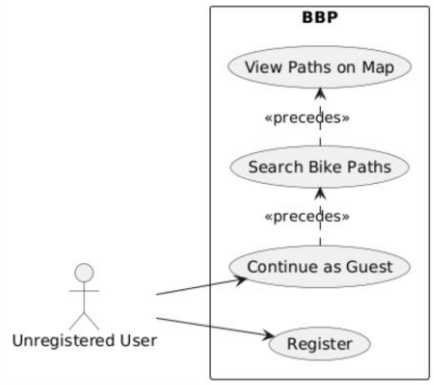


Figure 11 - Unregistered User Abilities

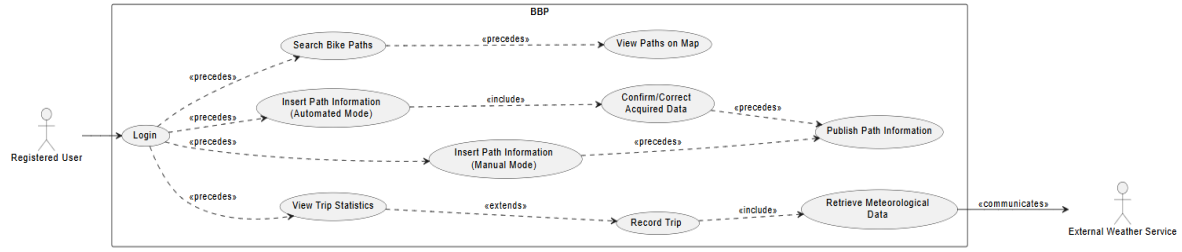


Figure 12 - Registered User Abilities

3.2.3. Use Cases

1. Login

Actors:	RU
Entry Conditions:	The RU is registered to BBP
Flow of Events:	1) RU opens the BBP application 2) BBP displays the home screen with login option 3) RU selects the login option 4) BBP displays the login form 5) RU enters their email and password 6) BBP forward the credentials to the database 7) The database validates the credentials 8) BBP returns a session token to the RU
Exit condition:	RU has logged in successfully to BBP
Exceptions:	1. The credentials are incorrect 2. BBP generated an internal error

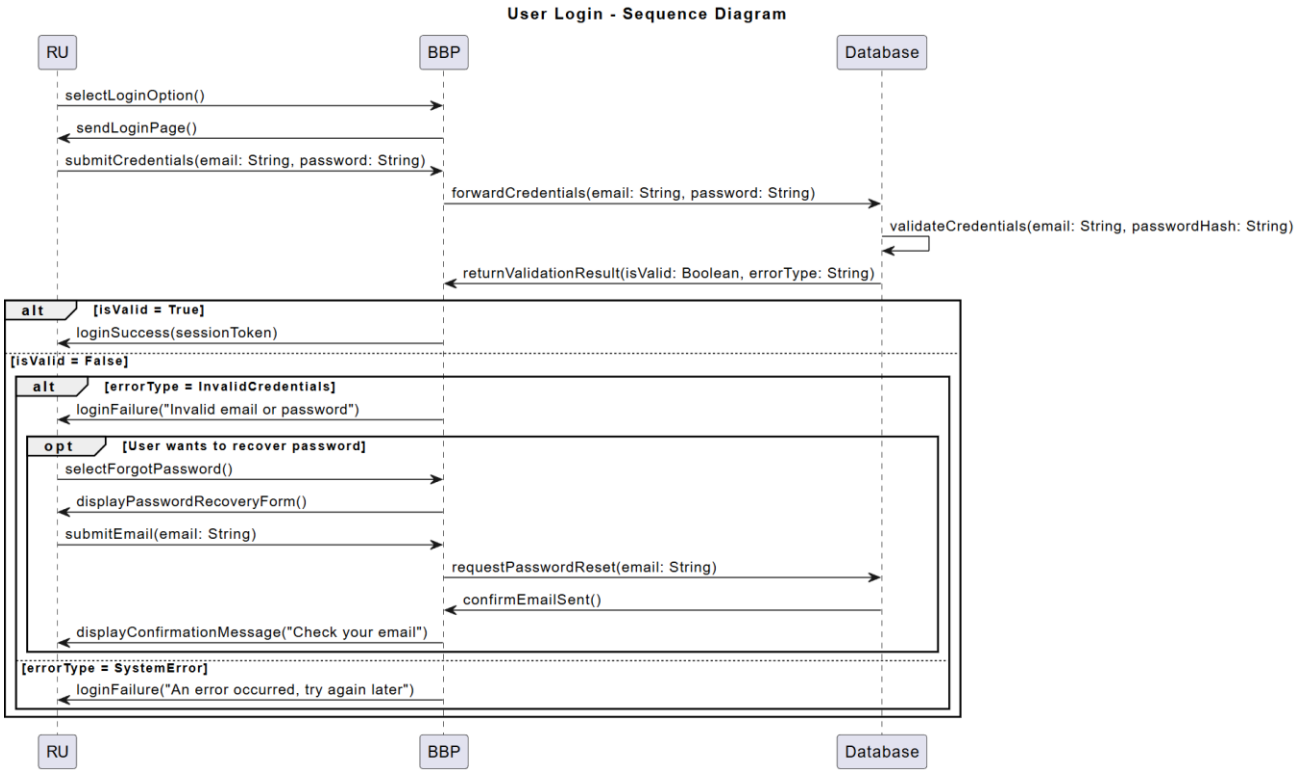


Figure 13 - Sequence diagram for login

2. Registration

Actors:	U
Entry Conditions:	The user is not yet registered in BBP
Flow of Events:	1) U opens the BBP application 2) BBP displays home screen with registration option 3) U selects the registration option 4) BBP displays the registration form 5) U enters their email, password and name 6) BBP forwards the data to the database 7) The database checks if the email already exists 8) BBP sends a verification email with a confirmation 9) RU clicks the verification link received via email 10) The database creates the user account 11) BBP displays the permissions configuration screen 12) RU configures permission preferences 13) The database saves the permission preferences 14) BBP confirms successful registration
Exit condition:	U has successfully registered to BBP
Exceptions:	1. The email is already registered 2. The verification email is not received or expired

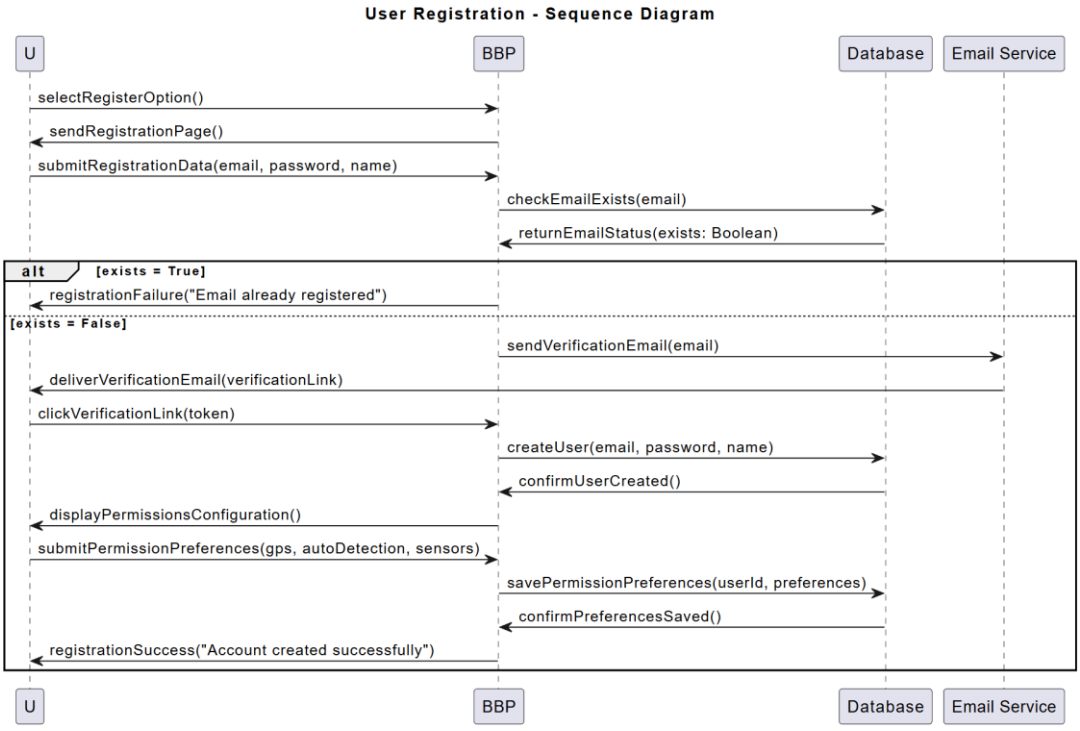


Figure 14 - Sequence diagram for registration

3. RU start / end recording

Actors:	RU
Entry Conditions:	The user is logged in to BBP and has granted GPS permission (or will grant it during the process)
Flow of Events:	1) RU taps the "Start Recording" button 2) BBP checks GPS permission status 3) If GPS permission was not granted during registration, BBP requests GPS permission 4) BBP initializes GPS tracking 5) BBP checks if sensors are enabled in preferences 6) If sensors are enabled, BBP initializes sensor collection for obstacle detection 7) During the trip, BBP continuously collects GPS data and sensor data (if sensors are active) 8) RU taps the "Stop Recording" button 9) BBP stops tracking and calculates trip statistics 10) The database stores the trip data temporarily 11) BBP displays the trip summary with statistics 12) BBP displays options to review/edit or discard
Exit condition:	RU has completed trip recording and chosen either to review/edit or discard the trip data
Exceptions:	1. GPS permission is denied (recording cannot proceed) 2. Sensors are disabled (recording proceeds without obstacle detection) 3. BBP generated an internal error during recording

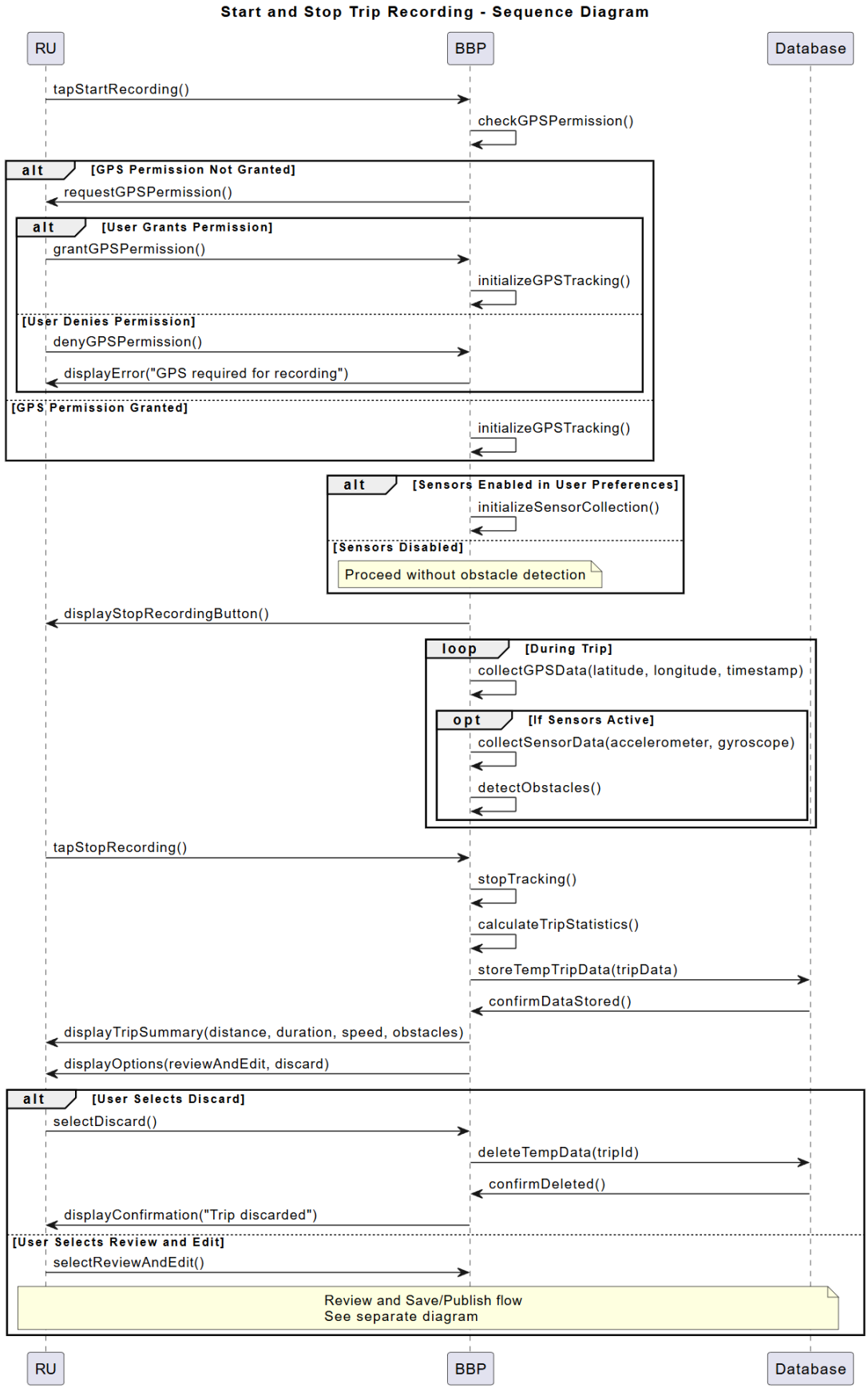


Figure 15 - Sequence diagram for trip recording

4. Search and visualize path on map

Actors:	U
Entry Conditions:	U
Flow of Events:	1) U accesses the path search feature 2) BBP displays the search interface 3) U enters origin and destination points 4) BBP sends the search request to the path search service 5) BBP queries the database for available bike paths 6) The database returns matching bike paths 7) BBP ranks paths by their quality score, if multiple paths are found 8) BBP displays a ranked list of paths scores 9) U selects "View on Map" 10) BBP sends the paths to the map service 11) The map service returns all bike paths on an interactive map 12) BBP displays the map to the U
Exit condition:	U has completed the search and visualized the result
Exceptions:	1. No paths are available for the selected search 2. BBP generated an internal error

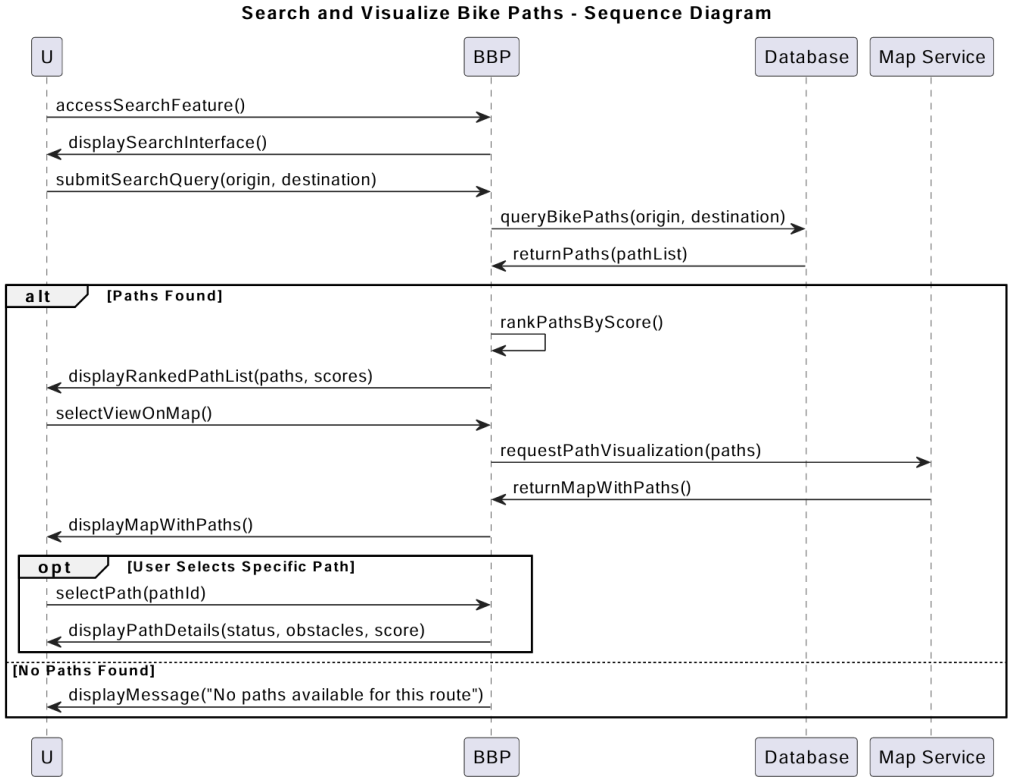


Figure 16 - Sequence diagram for search and visualize paths

5. Visualize My Trips

Actors:	RU
Entry Conditions:	RU is logged in to BBP
Flow of Events:	1) RU navigates to the "My Trips" 2) BBP retrieves the list of all trips recorded from the database 3) The database returns the trips list 4) BBP displays the trip history in chronological order (most recent first). Each trip entry shows basic information: date, distance, and duration 5) RU selects a specific trip to view detailed information 6) BBP retrieves the detailed trip data from the database 7) BBP displays all the detailed information about the trip: trip statistics (date, time, distance, speed, duration, elevation) and all path data (origin, destination, route) 8) BBP displays the route showing the path travelled 9) If weather information was stored with the trip, BBP displays the meteorological data (temperature, wind speed, weather conditions) 10) RU can return to the trip list
Exit condition:	RU has viewed their trip details and returned to the list
Exceptions:	1. No trips have been recorded yet 2. BBP generated an internal error

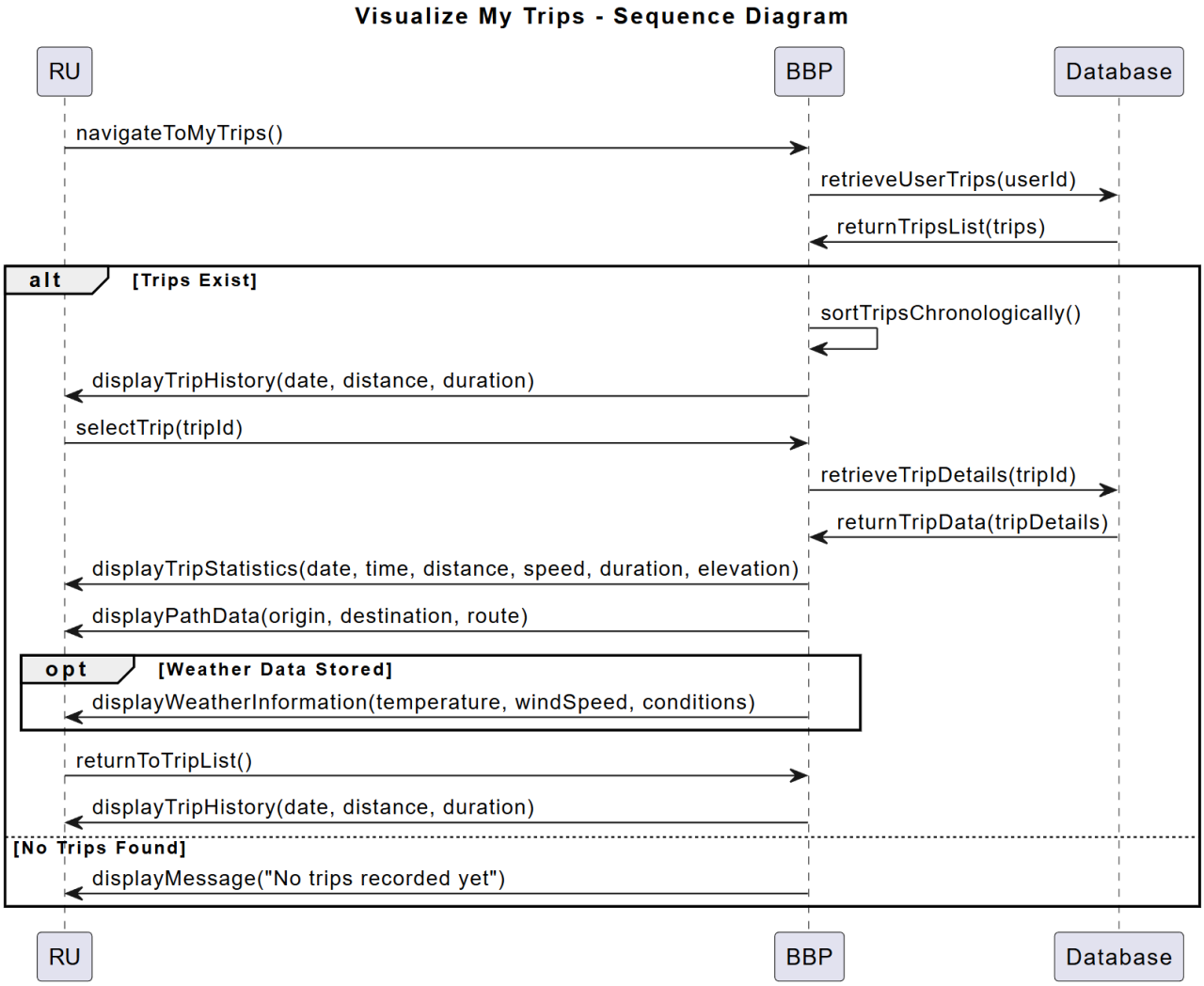


Figure 17 - Sequence diagram for visualizing My Trips

6. Automated detected path

Actors:	RU
Entry Conditions:	RU has automated detection mode enabled and has granted GPS permission. Sensor permissions are optional
Flow of Events:	1) BBP requests speed data from the device 2) The device returns the current speed 3) BBP analyses the speed to detect biking activity 4) When biking is detected, BBP requests the device to start GPS tracking 5) If sensors are enabled, BBP requests the device to start sensor collection 6) During the trip, BBP continuously requests GPS data from the device 7) If sensors are active, BBP requests accelerometer and gyroscope data from the device 8) BBP processes the data and detects obstacles 9) When speed decreases below threshold, BBP requests the device to stop tracking 10) BBP calculates trip statistics 11) The database stores the trip data temporarily 12) BBP sends a notification to RU about the detected path 13) RU opens the notification to view the trip summary 14) BBP displays the trip summary with statistics 15) BBP displays options to review/edit or discard
Exit condition:	RU has chosen to either discard or review/edit the detected trip
Exceptions:	1. BBP generated an internal error

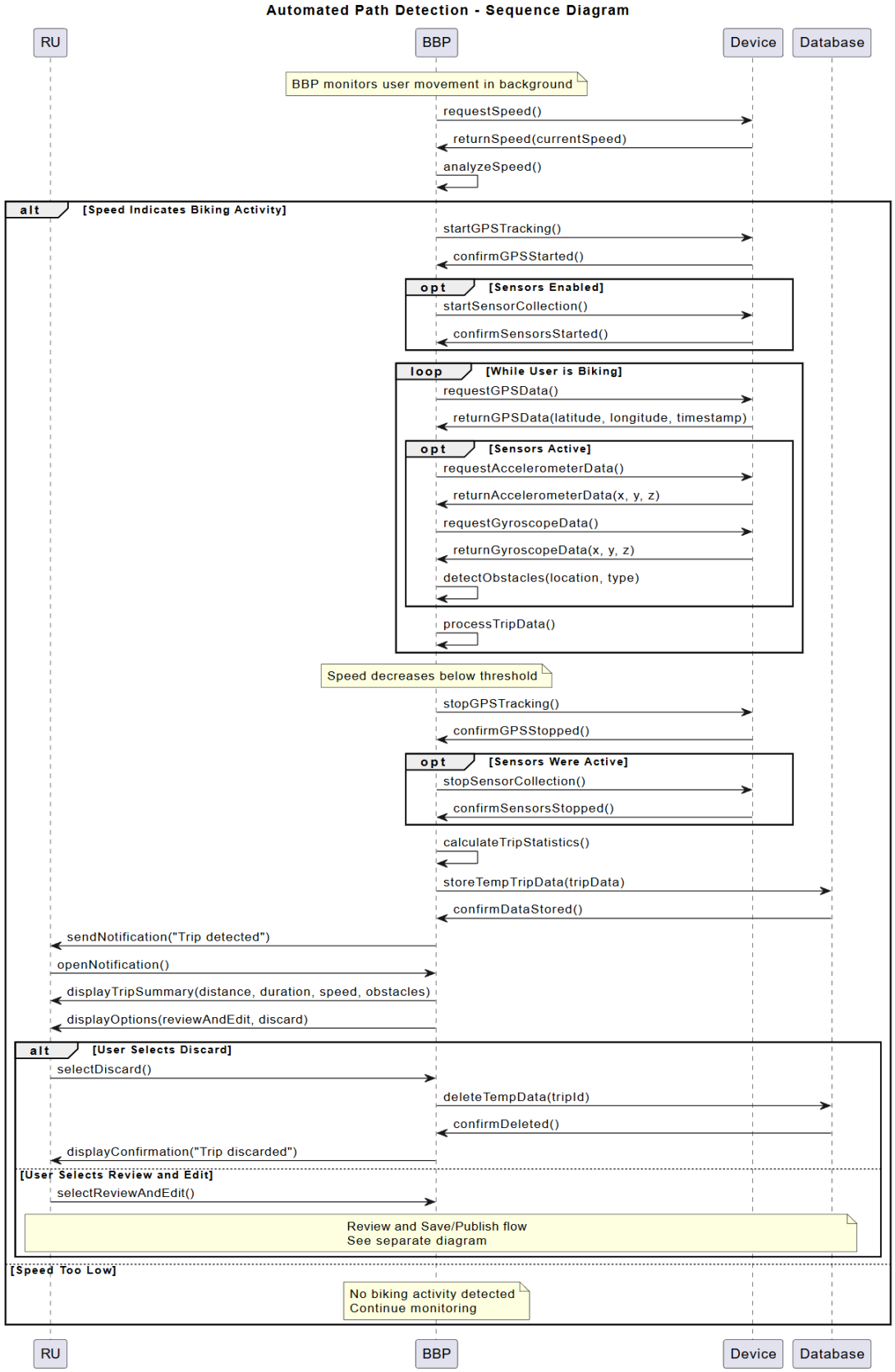


Figure 18 - Sequence diagram for automated detection

7. Insert path information manually

Actors:	RU
Entry Conditions:	RU is logged in to BBP
Flow of Events:	1) RU accesses the manual insertion feature in BBP 2) BBP displays manual insertion form 3) RU enters the street names in the sequence they followed during their bike trip 4) For each street, BBP verifies it exists in the map service 5) RU sets the path status (optimal, medium, sufficient, requires maintenance). 6) RU optionally adds information about obstacles with their locations 7) RU submits the path information 8) BBP validates that all required fields are filled 9) BBP displays a summary of the entered path information 10) RU reviews the information and confirms 11) The system proceeds with Save/Publish Options flow
Exit condition:	RU has chosen to either discard or review/edit the detected trip
Exceptions:	1. Street name is not found in the map service 2. Required fields are missing or data format is incorrect 3. BBP generated an internal error

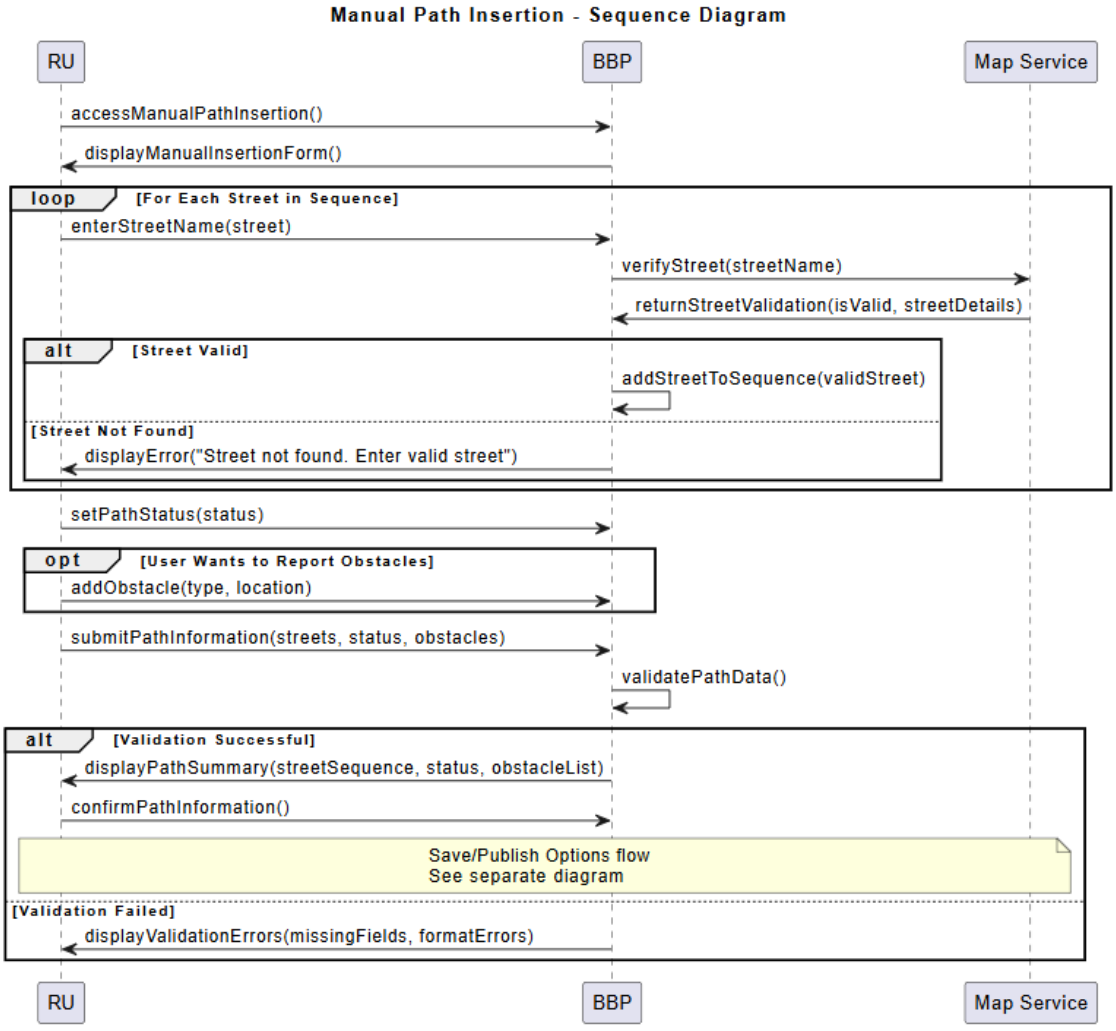


Figure 19 - Sequence diagram for path insertion

8. Review and Save/Publish Options

Actors:	RU
Entry Conditions:	RU is logged in and has path data from one of the following sources: manual recording, automatic detection, manual insertion (only for the save/publish flow from 5))
Flow of Events:	1) BBP displays the review screen showing all the path information 2) RU reviews and optionally modifies origin, destination, streets, obstacles, and path status 3) RU confirms the information 4) BBP updates the information based on the user's action 5) BBP displays three save/publish options to the user 6) RU selects one option: Save as Trip, Publish as Bike Path, or Save & Publish 7) If publishing is selected, BBP calculates the path score based on obstacles and status 8) The database stores and/or publishes the data based on the selected option 9) BBP confirms the operation to RU
Exit condition:	RU has saved and/or published the path information
Exceptions:	1. Network connection unavailable when publishing 2. BBP generated an internal error

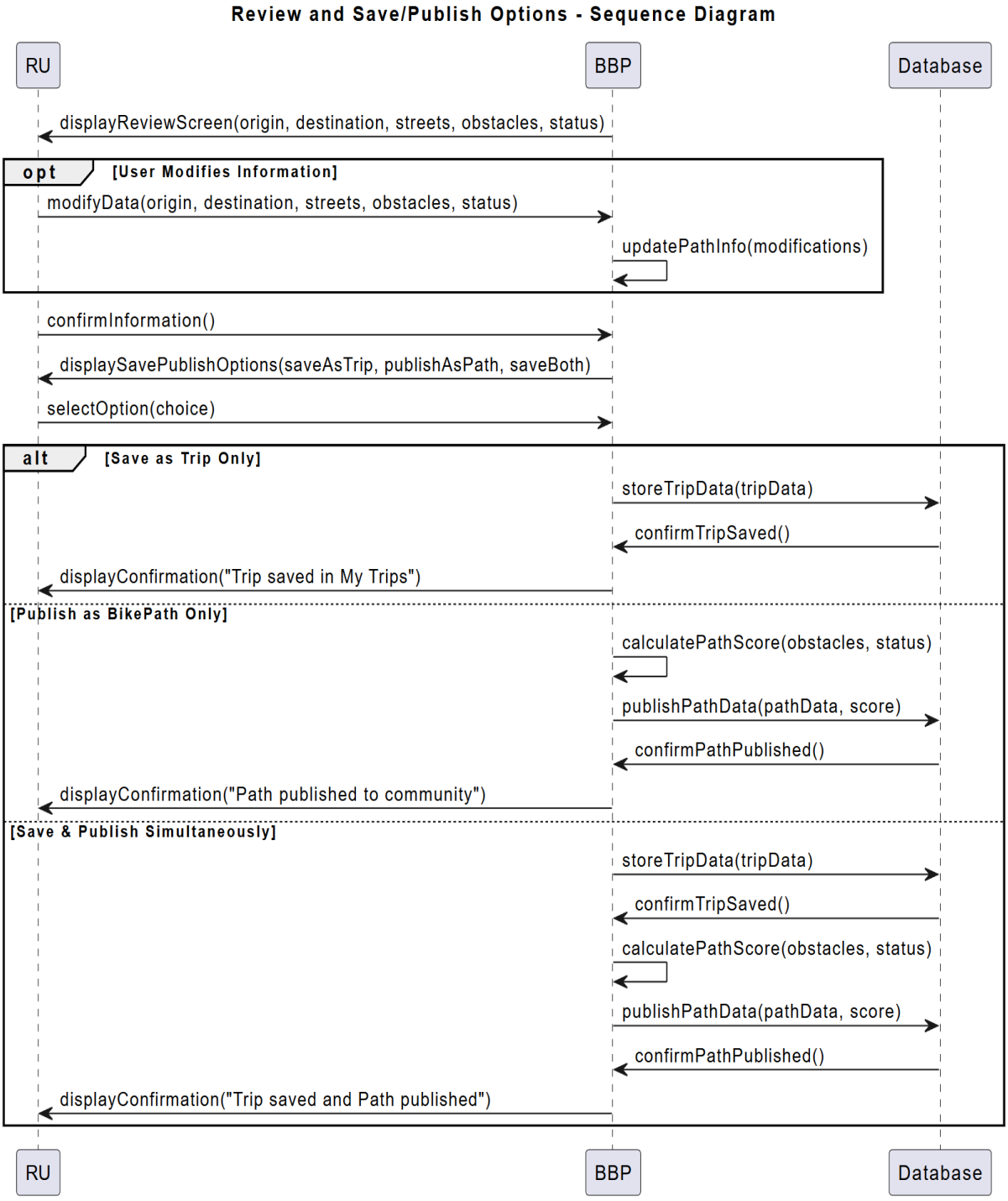


Figure 20 - Sequence diagram for Review and Save/Publish Options

8. Weather enriching information

Actors:	BBP
Entry Conditions:	RU has chosen to save trip data (from manual recording, automatic detection, or manual insertion)
Flow of Events:	1) BBP extracts the trip's timestamp (date and time) and GPS location from the recorded data 2) BBP sends a request to the external weather service with the trip's timestamp and location coordinates 3) The weather service retrieves weather data (temperature, wind speed, weather conditions) for that specific time and location 4) BBP associates the weather data with the trip 5) The database stores the trip data together with the meteorological information
Exit condition:	Trip data has been stored with or without weather information
Exceptions:	1. The weather service is unavailable (trip is saved without weather data) 2. BBP generated an internal error

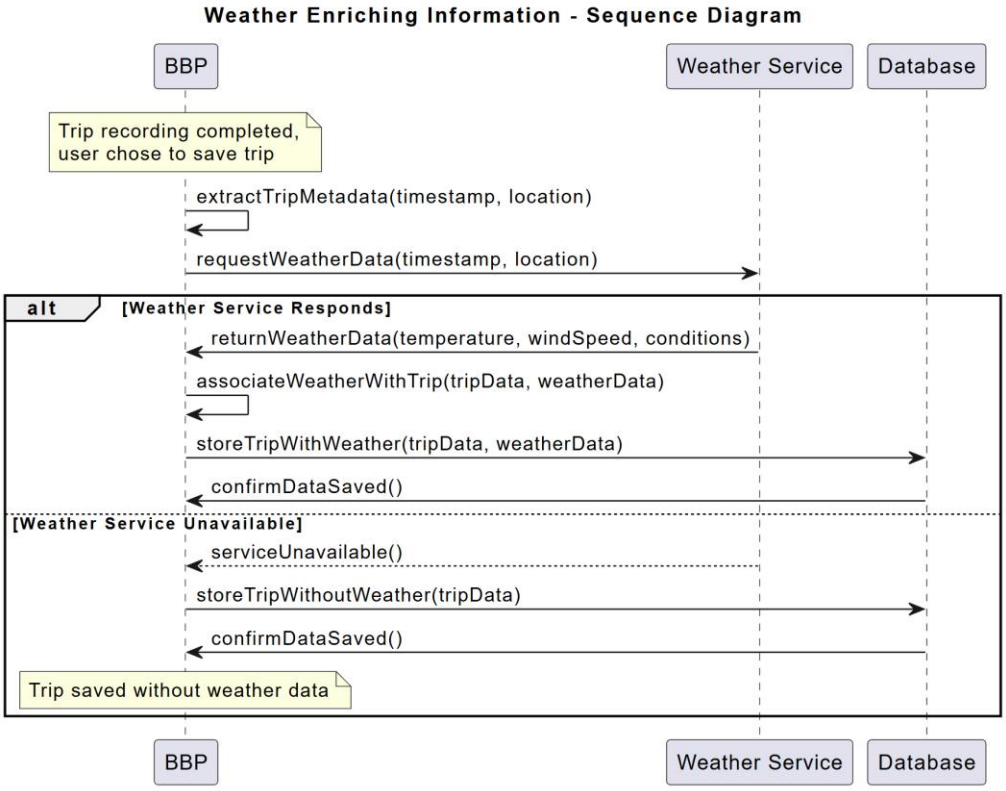


Figure 21 - Sequence diagram for weather enrichment

3.2.4. Goals-Requirements Mapping

$G_t \rightarrow$	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
R01	✓																											
R02		✓																										
R03					✓																							
R04			✓	✓																								
R05			✓	✓																								
R06						✓																						
R07						✓																						
R08							✓																					
R09								✓																				
R10									✓																			
R11										✓																		
R12											✓																	
R13								✓			✓																	
R14									✓		✓																	
R15																	✓	✓										
R16																	✓	✓										
R17																			✓									
R18																				✓								

G_i →	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
R19																				✓								
R20																					✓						
R21										✓																		
R22								✓		✓																		
R23									✓	✓																		
R24										✓																		
R25												✓					✓											
R26												✓													✓			
R27												✓													✓			
R28												✓													✓			
R29												✓										✓			✓			
R30												✓													✓			
R31													✓										✓					✓
R32														✓						✓				✓		✓		
R33															✓										✓			✓
R34													✓					✓									✓	
R35																		✓										
R36			✓	✓										✓					✓				✓			✓		
R37		✓		✓										✓					✓				✓			✓		

Table 7 – Goals-Requirements map

3.3. Requirements

Latency Requirements

BBP must be designed with a scalable and high-performance architecture to guarantee an excellent Quality of Experience for its users. In particular, the latency between a user's interaction and the system's response should remain below 300 ms, excluding end-to-end network delays that are not attributable to BBP itself. This requirement applies both to interactive features—such as browsing bike paths or viewing statistics—and to real-time data acquisition during automatic trip recording.

Storage Requirements

The system must safely store all collected data and implement proper disaster-recovery mechanisms. Stored information mainly consists of text-based records (trip data, path descriptions, obstacle reports) and sensor-generated data from mobile devices (GPS traces, accelerometer and gyroscope readings).
An average of 15–20 MB per registered user should be considered sufficient, making use of compression techniques when needed to ensure long-term scalability.

Architecture Sizing

BBP is targeted at the national cycling community. In countries with active cycling populations—such as Italy, with an estimated 8–10 million regular cyclists—reaching even **5%** of this audience would result in approximately 400k–500k potential users. Designing the system to support at least 500k active users will provide adequate capacity for expected adoption. The infrastructure should be dynamically scalable, able to handle traffic peaks and automatically adjust to support up to twice the monthly average user load, ensuring long-term flexibility, reliability, and performance.

3.4. Design Constraints

3.4.1. Standards compliance

BBP must comply with the General Data Protection Regulation (GDPR) to operate legally within the European Union. The system handles personal data including user credentials, GPS location data, and trip history, requiring strict adherence to data protection principles. User consent must be explicitly obtained for data collection, particularly for GPS tracking and sensor data acquisition. The system must implement appropriate technical and organizational measures to ensure data security and privacy.

3.4.2. Hardware limitations

Client-side, BBP requires mobile devices running modern versions of iOS or Android with GPS capability. For automatic obstacle detection functionality, devices must include accelerometer and gyroscope sensors. Server-side, the backend infrastructure must be capable of handling concurrent requests from multiple users and storing GPS trace data efficiently.

3.4.3. Any Other Constraint

The system relies on native mobile mapping services for geocoding and map visualization, requiring compliance with their respective terms of service and API usage limits. Weather data enrichment depends on the availability and reliability of native weather APIs provided by the mobile operating systems. The automatic cycling detection and continuous GPS tracking features have significant impact on device battery consumption. The system must implement efficient background processing to minimize battery consumption.

3.5. Software System Attributes

3.5.1. Reliability

The system must be designed so that faults cannot propagate across the entire architecture, and errors can be quickly identified and isolated. Sufficient replication of critical data—such as trip records, GPS traces, and published path information—and of core services should be implemented to ensure high reliability. This guarantees that BBP remains operational even in the presence of component failures, particularly during real-time sensor data collection and path-browsing operations.

3.5.2. Availability

The system must maintain an availability target of 99.5%, allowing approximately 44 hours of downtime per year. Planned maintenance should be scheduled during low-traffic periods with advance user notification. The backend must implement failover mechanisms and data replication to ensure service continuity. Some functionalities such as trip recording can operate offline, with automatic synchronization when connectivity is restored.

3.5.3. Security

BBP must implement a comprehensive authentication and authorization framework to manage user access rights. Authentication verifies the identity of registered users, while authorization ensures that only permitted actions—such as publishing path information or viewing My Trips —can be performed.

To protect the system's integrity, standard security measures will be enforced, including encryption of user credentials and personal data, secure handling of sensor-generated trip data, and protection mechanisms against injection attacks and other common security vulnerabilities.

3.5.4. Maintainability

The system must adopt a modular architecture with clear separation between mobile applications, backend services, and external integrations. Code must follow consistent standards and be well-documented to facilitate debugging and feature development. The infrastructure should support rolling updates to enable deployments without downtime. Logging and monitoring mechanisms must be implemented to quickly identify and resolve issues.

3.5.5. Portability

BBP's server-side architecture must be able to run on standard x86 hardware. The client-side application must be accessible from any modern browser on a variety of endpoints, including smartphones, tablets, and smartwatches, ensuring full compatibility across mobile devices equipped with GPS and sensor capabilities.

4. Formal analysis using Alloy

This section presents the formal analysis and verification of the system. The analysis is done using the Alloy language and the Alloy Analyzer tool.

4.1 Alloy model

```
// --- Users ---
sig User {}
sig RegisteredUser extends User {
    usedDevice: lone MobileDevice,
    createdPathInfos: set PathInfo,
    ownedTrips: set Trip
}

// --- Mobile Device and Sensors ---
sig MobileDevice {
sensorsEnabled: one Bool,
autoDetectionEnabled: one Bool,
GPSEnabled: one Bool,
producedData: set AcquiredPathData
}

// --- Acquisition Mode ---
abstract sig AcquisitionMode {}
one sig MANUAL_RECORDING, AUTOMATIC_DETECTION extends AcquisitionMode {}

// --- Path Status ---
abstract sig PathStatus {}
one sig OPTIMAL, MEDIUM, SUFFICIENT, REQUIRES_MAINTENANCE extends PathStatus {}

// --- Obstacle Type ---
abstract sig ObstacleType {}
one sig POTHOLE, DEBRIS, CONSTRUCTION, OTHER extends ObstacleType {}
```

```
// --- Acquired Path Data---
sig AcquiredPathData {
      collectedBy: one RegisteredUser,
      mode: one AcquisitionMode,
      detectedObstacles: set Obstacle,
      suggestedStatus: one PathStatus,
      reviewedTo: lone PathInfo
      //after an acquired path data has been reviewed it became a Path Info
}

// --- Path Info ---
sig PathInfo {
    owner: one RegisteredUser,
    isPublished: one Bool,
    status: one PathStatus,
    segments: some PathSegment,  // At least one segment
    savedAsTrip: lone Trip,  // Can generate at most one Trip
    publishedAsBikePath: lone BikePath  // Can be published at most once
//A path info can become  - a trip if it is saved only for the user
//                        - or a bike path if it is published
}

// --- Trip ---
sig Trip {
    belongsTo: one RegisteredUser,
    sourcePathInfo: one PathInfo,
    publishedAsBikePath: lone BikePath,
    enrichedWeather: lone WeatherData
// In any moment the user can decide to publish a trip as BikePath
}

// --- Bike Path ---
sig BikePath {
    sourcePathInfo: one PathInfo,
    contributor: one RegisteredUser,
    displayedSegments: some PathSegment
}
```

```
// --- Path Segment ---
sig PathSegment {
    obstacles: set Obstacle  // Can have zero or more obstacles
}

// --- Obstacle ---
sig Obstacle {
    obstacleType: one ObstacleType,
    confirmed: one Bool,
}

// --- Weather Data ---
sig WeatherData {}

// --- Boolean Type ---
abstract sig Bool {}
one sig True, False extends Bool {}
```

Facts of the model:

```
// --- Registered User Consistency ---
fact RUConsistency {
    // only registered user can create a path info
    all u: RegisteredUser, p: PathInfo | p in u.createdPathInfos iff u = p.owner
    // only registered user can own a trip
   all u: RegisteredUser, t: Trip |t in u.ownedTrips iff u = t.belongsTo
    // Device consistency ownership
    all u: RegisteredUser, d: MobileDevice, a: AcquiredPathData | (u.usedDevice
= d and a in d.producedData) implies a.collectedBy = u
}

// --- PathInfo and Trip Consistency ---
fact PathInfoTripConsistency {
    // Bidirectional PathInfo <-> Trip
    all p: PathInfo, t: Trip | t = p.savedAsTrip iff t.sourcePathInfo = p
    // Trip owner must match PathInfo owner
    all t: Trip | t.belongsTo = t.sourcePathInfo.owner
}
```

```
// --- BikePath Constraints ---
fact BikePathConstraints {
    // Bidirectional PathInfo <-> BikePath
    all p: PathInfo, b: BikePath|b = p.publishedAsBikePath iff b.sourcePathInfo
= p
    // BikePath segments must be from sourcePathInfo
    all b: BikePath | b.displayedSegments in b.sourcePathInfo.segments
    // BikePath contributor must be PathInfo owner
    all b: BikePath | b.contributor = b.sourcePathInfo.owner
    // Published flag consistency
    all p: PathInfo |p.isPublished = True iff some p.publishedAsBikePath
}

// --- Obstacle Confirmation ---
fact ObstacleConfirmation {
    // Obstacles in PathInfo segments are confirmed
    all p: PathInfo, s: PathSegment, o: Obstacle |(s in p.segments and o in
s.obstacles) implies o.confirmed = True
    // Unreviewed obstacles in AcquiredPathData are unconfirmed
    all a: AcquiredPathData, o: Obstacle |(o in a.detectedObstacles and no
a.reviewedTo) implies o.confirmed = False
}

// --- Review Consistency ---
fact ReviewConsistency {
    // If reviewed, PathInfo owner must be the collector
    all a: AcquiredPathData, p: PathInfo |p = a.reviewedTo implies p.owner =
a.collectedBy
}

// --- GPS Requirement for Data Acquisition ---
fact GPSRequirementForAcquisition {
    // Both manual recording and automatic detection require GPS enabled
    all a: AcquiredPathData, d: MobileDevice |
        (a in d.producedData) implies d.GPSEnabled = True
}
```

```
// --- Obstacle Existence Constraint ---
fact ObstacleExistence {
    // An obstacle can be confirmed only if it exists in the model
    // This means: if an obstacle is confirmed, it must be referenced somewhere
    all o: Obstacle | o.confirmed = True implies (
        (some a: AcquiredPathData | o in a.detectedObstacles) or
        (some s: PathSegment | o in s.obstacles)
    )
}
```

Useful assertions, predicates and functions used in the model itself and for testing it during the development:

```
// --- Manual Recording Workflow ---
pred manualRecordingWorkflow[u: RegisteredUser, d: MobileDevice, a: AcquiredPathData] {
    u.usedDevice = d
    a in d.producedData
    a.mode = MANUAL_RECORDING
    a.collectedBy = u
}

// --- Automatic Detection Workflow ---
pred automaticDetectionWorkflow[u: RegisteredUser, d: MobileDevice, a: AcquiredPathData] {
    u.usedDevice = d
    d.autoDetectionEnabled = True
    a in d.producedData
    a.mode = AUTOMATIC_DETECTION
    a.collectedBy = u
}

// --- Complete Publication Flow ---
pred completePublicationFlow[u: RegisteredUser, a: AcquiredPathData, p: PathInfo, b: BikePath] {
    a.reviewedTo = p
    p.publishedAsBikePath = b
    u = p.owner
}
```

```
// --- Complete Trip Flow ---
pred completeTripFlow[u: RegisteredUser, a: AcquiredPathData, p: PathInfo, t: Trip, b: BikePath] {
    a.reviewedTo = p
    p.savedAsTrip = t
    t.publishedAsBikePath = b
    u = p.owner
}
// --- Predicate: Check GPS is enabled for manual recording ---
pred checkGPSForManualRecording[d: MobileDevice, a: AcquiredPathData] {
    a in d.producedData
    a.mode = MANUAL_RECORDING
    d.GPSEnabled = True
}

// --- Predicate: Check GPS is enabled for automatic detection ---
pred checkGPSForAutomaticDetection[d: MobileDevice, a: AcquiredPathData] {
    a in d.producedData
    a.mode = AUTOMATIC_DETECTION
    d.GPSEnabled = True
}

// --- Show GPS requirement for manual recording ---
pred showGPSManualRecording {
    some d: MobileDevice, a: AcquiredPathData |
        checkGPSForManualRecording[d, a]
}

// --- Show GPS requirement for automatic detection ---
pred showGPSAutomaticDetection {
    some d: MobileDevice, a: AcquiredPathData |
        checkGPSForAutomaticDetection[d, a]
}

// --- Manual Insertion Workflow ---
pred manualInsertionWorkflow[u: RegisteredUser, p: PathInfo] {
    // PathInfo created directly without AcquiredPathData
```

```
    p.owner = u
    no a: AcquiredPathData | a.reviewedTo = p
    // PathInfo has segments and can have already confirmed obstacles
    some p.segments
    all s: p.segments, o: Obstacle |
        (o in s.obstacles) implies o.confirmed = True
}

// --- Obstacle Confirmation Flow ---
pred obstacleConfirmationFlow[a: AcquiredPathData, p: PathInfo] {
    // Show the obstacle confirmation process
    a.reviewedTo = p

    // Obstacles in unreviewed AcquiredPathData are unconfirmed
    all o: Obstacle | o in a.detectedObstacles and no a.reviewedTo
        implies o.confirmed = False

    // After review, obstacles in PathInfo are confirmed
    all s: p.segments, o: Obstacle |
        (s in p.segments and o in s.obstacles) implies o.confirmed = True
}

// --- All obstacles in PathInfo are confirmed ---
assert PathInfoObstaclesAreConfirmed {
    all p: PathInfo, s: PathSegment, o: Obstacle |
        (s in p.segments and o in s.obstacles) implies o.confirmed = True
}

// --- PathInfo from AcquiredPathData ownership consistency ---
assert ReviewedPathInfoHasAcquiredData {
    all p: PathInfo, a: AcquiredPathData |
        a.reviewedTo = p implies p.owner = a.collectedBy
}

// --- Assertion: Manual insertion creates valid PathInfo ---
assert ManualInsertionCreatesValidPathInfo {
    all p: PathInfo |
```

```
        (no a: AcquiredPathData | a.reviewedTo = p) implies
        (some p.owner and some p.segments)
}

// --- Assertion: Obstacle confirmation consistency ---
assert ObstacleConfirmationConsistency {
    all o: Obstacle, a: AcquiredPathData |
        (o in a.detectedObstacles and no a.reviewedTo) implies o.confirmed = False
}

// --- Show a basic scenario with all entities ---
pred showBasicScenario {
    some RegisteredUser
    some MobileDevice
    some AcquiredPathData
    some PathInfo
    some Trip
    some BikePath
    some PathSegment
    some Obstacle
}

// --- Show manual recording workflow ---
pred showManualRecording {
    some u: RegisteredUser, d: MobileDevice, a: AcquiredPathData |
        manualRecordingWorkflow[u, d, a]
}

// --- Show automatic detection workflow ---
pred showAutomaticDetection {
    some u: RegisteredUser, d: MobileDevice, a: AcquiredPathData |
        automaticDetectionWorkflow[u, d, a]
}

// --- Show complete publication flow ---
pred showPublicationFlow {
    some u: RegisteredUser, a: AcquiredPathData, p: PathInfo, b: BikePath |
```

```
        completePublicationFlow[u, a, p, b]
}

// --- Show complete trip flow ---
pred showTripFlow {
    some u: RegisteredUser, a: AcquiredPathData, p: PathInfo, t: Trip, b: BikePath
|
        completeTripFlow[u, a, p, t, b]
}

// --- Show manual insertion scenario ---
pred showManualInsertion {
    some u: RegisteredUser, p: PathInfo |
        manualInsertionWorkflow[u, p] and
        // At least one segment with a confirmed obstacle
        some s: p.segments, o: Obstacle |
            s in p.segments and o in s.obstacles
}

// --- Show obstacle confirmation flow ---
pred showObstacleConfirmation {
    some a: AcquiredPathData, p: PathInfo |
        obstacleConfirmationFlow[a, p] and
        // There are detected obstacles that get confirmed
        some a.detectedObstacles and
        some s: p.segments | some s.obstacles
}

// --- Show complete obstacle lifecycle ---
pred showObstacleLifecycle {
    // Show complete cycle: detection -> review -> confirmation
    some u: RegisteredUser, d: MobileDevice, a: AcquiredPathData, p: PathInfo,
o: Obstacle |
        // Obstacle detected during acquisition
        a in d.producedData and
        a.collectedBy = u and
        o in a.detectedObstacles and
        // Obstacle is initially unconfirmed
```

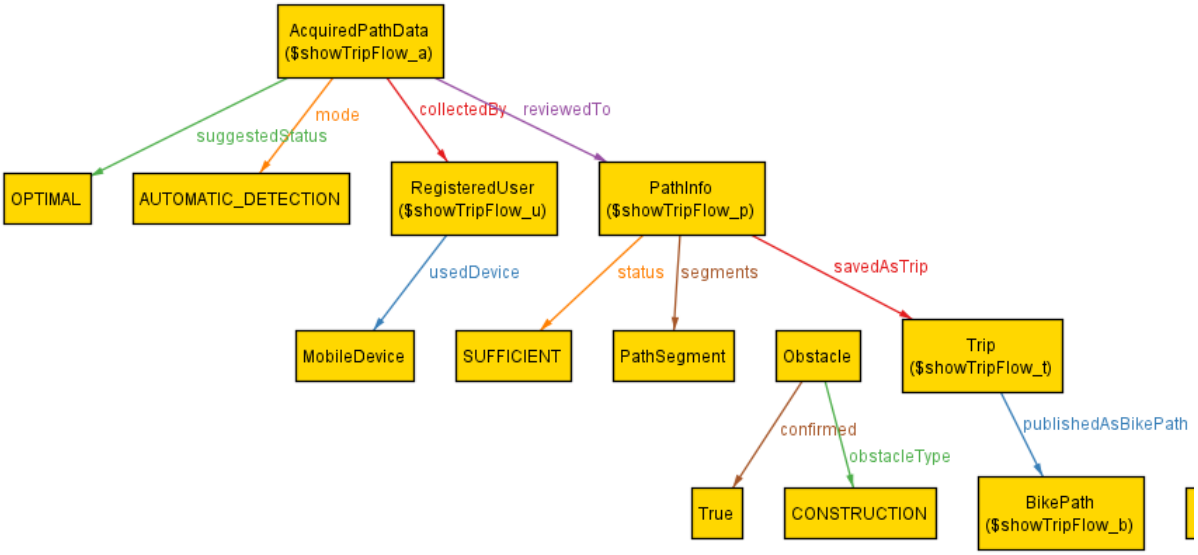
```
        (no a.reviewedTo implies o.confirmed = False) and
        // After review, obstacle becomes confirmed
        a.reviewedTo = p and
        some s: p.segments | o in s.obstacles and o.confirmed = True
}

// --- Run commands ---
run showBasicScenario for 5
run showManualRecording for 5
run showAutomaticDetection for 5
run showPublicationFlow for 5
run showTripFlow for 5
run showGPSManualRecording for 5
run showGPSAutomaticDetection for 5
run showManualInsertion for 5
run showObstacleConfirmation for 5
run showObstacleLifecycle for 5

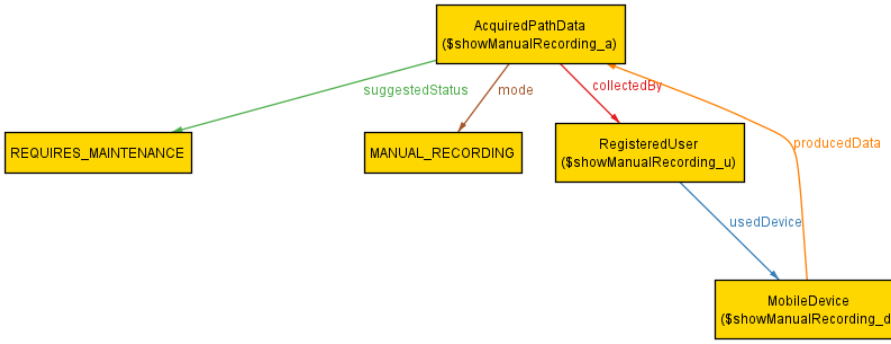

// --- Check assertions ---
check PathInfoObstaclesAreConfirmed for 5
check ReviewedPathInfoHasAcquiredData for 5
check ManualInsertionCreatesValidPathInfo for 5
check ObstacleConfirmationConsistency for 5
```

4.2 Alloy result

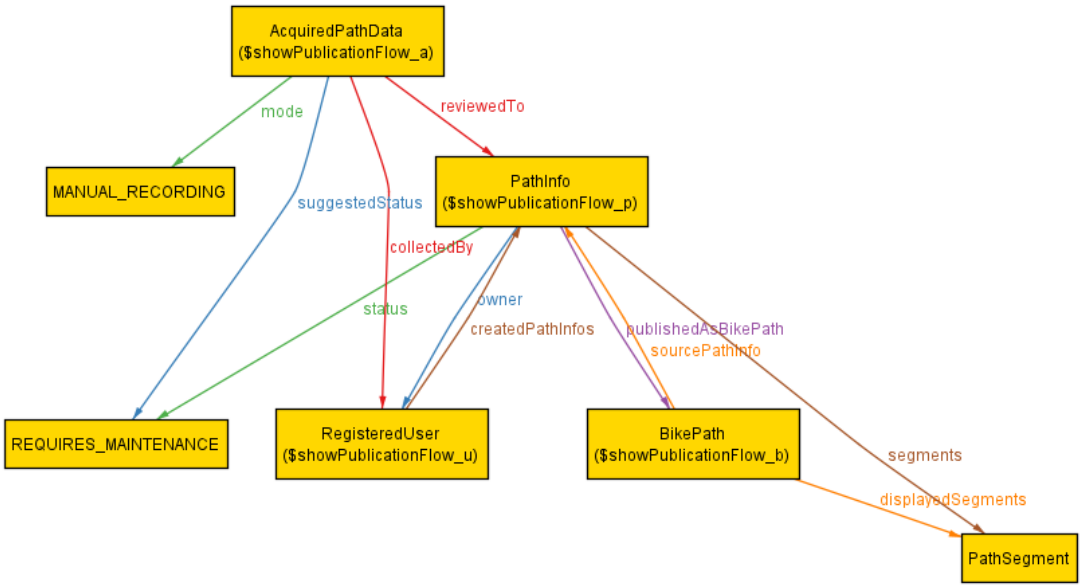
Automatic detection scenario:



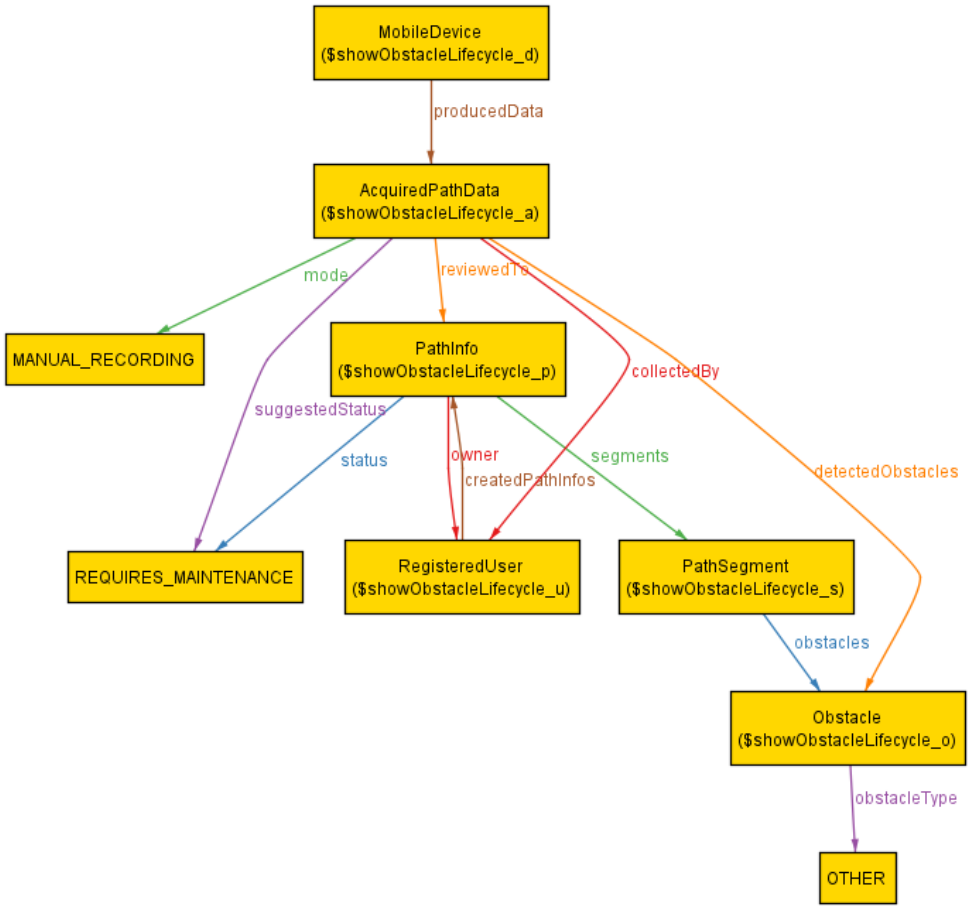
Manual recording scenario:



Publish flow scenario:



Obstacle detection scenario:



5. Effort spent

Student	Section	Hours
Anna Cordioli	*1*	5
	2	4
	3	5
	4	5
TOTAL		19 h
Tasnim Elkoush	*1*	5
	2	5
	3	6
	4	4
TOTAL		20 h

6. References

6.1. References

• ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering.

6.2. Used Tools

• GitHub - Used for project versioning and sharing.

• Word - Used as editor for writing the document.

- demo.bpmn.io/s/start - Used for state diagrams' design.
- plantuml-editor.kkeisuke.dev- Used for sequence diagrams' design.
- Alloy - Used for formal analysis.